FlashAttention 论文精读与全文解读

Fast and Memory-Efficient Exact Attention with IO-Awareness

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré
Stanford University & University at Buffalo, SUNY

NeurIPS 2022

精读整理

Abstract

本文档是对 FlashAttention 论文（arXiv:2205.14135）的全文精读与逐段解读。每一节包含：灰色框= 原文完整翻译，绿色框= 通俗解释，蓝色框= 数学推导详解，橙色框= 关键概念总结，红色框= 注意事项，以及行内【引用说明】。文末附有完整参考文献一览表。

Contents

1 摘要 (Abstract) **3**

2 引言 (Introduction) **4**

3 背景 (Background) **7**
3.1 硬件性能 (Hardware Performance) 7
3.2 标准注意力实现 (Standard Attention Implementation) 8

4 FlashAttention：算法、分析与扩展 **10**
4.1 使用分块与重计算的高效注意力算法 10
4.2 FlashAttention 算法伪代码 12
4.3 分析：FlashAttention 的 IO 复杂度 14
4.4 扩展：块稀疏 FlashAttention 17

5 实验 (Experiments) **17**
5.1 使用 FlashAttention 的更快模型 17
5.2 更长序列带来更好模型 18
5.3 注意力基准测试 20

6 局限与未来方向 (Limitations and Future Directions) **21**

7 附录 A：相关工作 (Related Work) **22**

8 附录 B：算法细节 **22**
8.1 内存高效的前向传播 22
8.2 FlashAttention 完整前向传播（算法 2） 23
8.3 FlashAttention 反向传播 24

9 附录 C：正确性与复杂度证明 **27**

10 附录 D：扩展细节 **28**
10.1 块稀疏 FlashAttention（算法 5） 28

11 附录 E：完整实验结果 **29**

12 附录 F：参考文献一览表 **34**

1 摘要 (Abstract)

原文翻译

Transformer 在长序列上速度慢且内存消耗大，因为自注意力（self-attention）的时间复杂度和内存复杂度关于序列长度都是二次的（$O(N^2)$）。近似注意力方法尝试通过在模型质量和计算复杂度之间做权衡来解决这个问题，但通常无法实现实际的墙钟时间（wall-clock time）加速。我们认为，一个缺失的关键原则是使注意力算法具备 **IO 感知性**（IO-aware）——即考虑 GPU 不同层级内存之间的读写操作。

我们提出 FlashAttention，一种 IO 感知的精确注意力算法，它使用**分块**（tiling）技术来减少 GPU 高带宽内存（HBM）与 GPU 片上 SRAM 之间的内存读写次数。我们分析了 FlashAttention 的 IO 复杂度，证明它比标准注意力需要更少的 HBM 访问次数，并且在一定的 SRAM 大小范围内是最优的。我们还将 FlashAttention 扩展到块稀疏注意力（block-sparse attention），得到一种比任何现有近似注意力方法都更快的近似注意力算法。

FlashAttention 能够使 Transformer 训练更快：在 BERT-large（序列长度 512）上比 MLPerf 1.1 训练速度记录快 15%；在 GPT-2（序列长度 1K）上获得 3 倍加速；在长程竞技场（Long-Range Arena，序列长度 1K–4K）上获得 2.4 倍加速。FlashAttention 和块稀疏 FlashAttention 使 Transformer 能够处理更长的上下文，从而得到更高质量的模型（GPT-2 困惑度提升 0.7，长文档分类提升 6.4 个点），并带来了全新的能力：首个在 Path-X 挑战（序列长度 16K，准确率 61.4%）和 Path-256（序列长度 64K，准确率 63.1%）上取得优于随机水平表现的 Transformer。

通俗解释

想象你有一个大班级，老师（Transformer）想让每个学生（token）都和其他所有学生交流一轮。如果班级有 N 个学生，那交流的次数就是 N^2——当 N 很大时（比如一篇长文章有几千甚至几万个词），这就变得极其缓慢和耗费内存。

之前的解决方案是「偷懒」——让学生只和部分同学交流（近似注意力），但这种偷懒虽然减少了计算量，实际速度却没快多少，因为它忽略了一个关键问题：**数据搬运**。GPU 里的高速小内存（SRAM，类似你书桌上的便签纸）容量很小但速度飞快，而大内存（HBM，类似书架上的书）容量大但拿取慢。如果你反复从书架取书又放回去，即使算得很快，搬运时间也会很长。

FlashAttention 的核心思想就是：把注意力计算分成小块（tiling），每次只搬运一小块数据到高速内存中完成计算，大幅减少了数据搬运次数。结果是，即使在数学计算量上没有减少（甚至略有增加），实际运行时间却大大缩短了。

核心要点

1. FlashAttention 是一种**精确**（非近似）的注意力算法，得到的结果与标准注意力完全相同。
2. 核心创新不是减少计算量（FLOPs），而是减少**内存访问次数**（IO）。
3. 使用两个关键技术：**分块**（tiling）和**重计算**（recomputation）。
4. 在实际 GPU 上实现了 2–7 倍的加速，内存使用从 $O(N^2)$ 降到 $O(N)$。

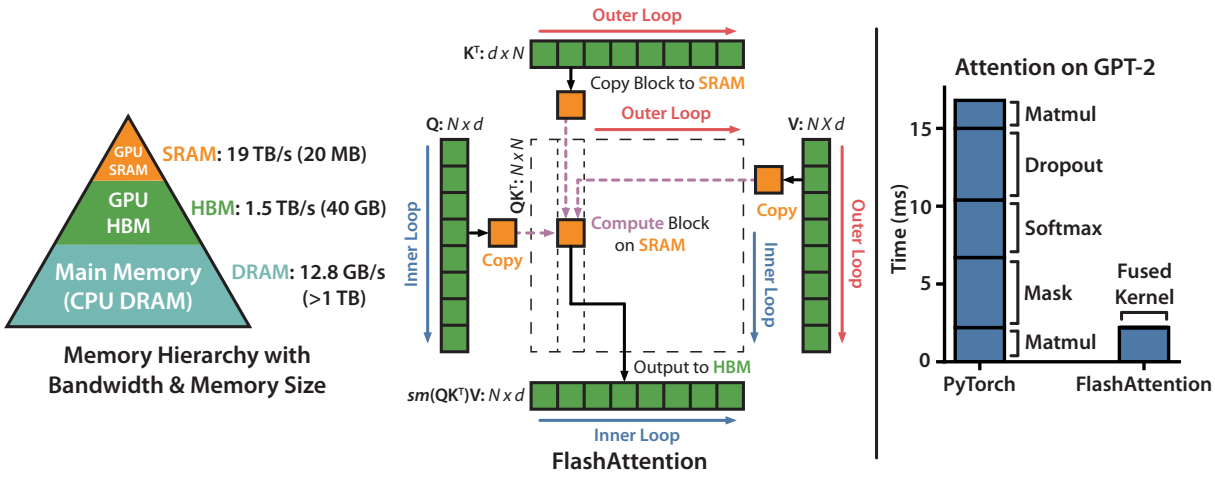

Figure 1: **图 1（论文原图）。左：**FlashAttention 使用分块（tiling）技术避免在（相对）慢速的 GPU HBM 中物化大型 $N \times N$ 注意力矩阵（虚线框）。在外循环中（红色箭头），FlashAttention 遍历 **K** 和 **V** 矩阵的块并将它们加载到快速片上 SRAM。在每个块中，FlashAttention 遍历 **Q** 矩阵的块（蓝色箭头），将它们加载到 SRAM，并将注意力计算的输出写回 HBM。**右：**在 GPT-2 上相对于 PyTorch 标准注意力实现的加速比。FlashAttention 不读写大型 $N \times N$ 注意力矩阵到 HBM，使注意力计算获得高达 7.6 倍加速。

2 引言 (Introduction)

图 1 解读

这是论文最重要的总览图，分为左右两部分：
左图——FlashAttention 的工作原理示意图：
图中展示了 GPU 的内存层次结构。最上方是大容量但慢速的 **HBM**（高带宽内存），下方是小容量但极快的 **SRAM**（片上静态随机存储器）。
标准注意力会把 **Q**、**K**、**V** 从 HBM 读到 GPU 计算单元，算出巨大的 $N \times N$ 注意力矩阵 **S** 后又写回 HBM，再读出来做 softmax，再写回去……反复搬运。图中虚线框标出的就是这个 $N \times N$ 矩阵——它在标准方法中需要完整存在于 HBM，是内存和速度瓶颈的根源。
FlashAttention 的做法是：

- **红色箭头**（外循环）：每次从 HBM 取出一小块 $\mathbf{K}_j$ 和 $\mathbf{V}_j$ 放到 SRAM。
- **蓝色箭头**（内循环）：在 SRAM 中，让每一小块 $\mathbf{Q}_i$ 与当前的 $\mathbf{K}_j$ 做乘法，计算局部注意力并增量更新输出。
- 整个过程中，$N \times N$ 的注意力矩阵**永远不会被完整地写入 HBM**——它只以小块形式短暂存在于 SRAM 中。

右图——GPT-2 上的加速效果：
横轴是序列长度（128 到 4096），纵轴是相对于 PyTorch 标准注意力的加速倍数。可以看到：

- 在所有序列长度上，FlashAttention 都显著快于标准注意力。
- 随着序列长度增加（即 $N \times N$ 矩阵变大），加速效果更加明显——在序列长度 4096 时达到 **7.6 倍**加速。
- 这验证了 IO 优化在长序列场景下效果更大的理论预测。

原文翻译

Transformer 模型【引用说明：Vaswani et al., 2017 —提出了 Transformer 架构的开创性论文，本文的研究对象正是 Transformer 中的注意力模块】已成为自然语言处理和图像分类等应用中最广泛使用的架构。Transformer 变得越来越大【引用说明：Brown et al., 2020 (GPT-3) —展示了超大语言模型的能力，说明 Transformer 规模在不断增长】、越来越深【引用说明：Wang et al., 2022 (DeepNet) —展示了更深的 Transformer 架构，说明深度也在增加】，但为它们配备更长的上下文仍然困难【引用说明：Tay et al., 2020 (Long Range Arena) —提出了长程建模基准测试，说明长序列处理是公认的难题】，因为其核心的自注意力模块的时间和内存复杂度关于序列长度是二次的。一个重要的问题是：使注意力更快、更节省内存是否能帮助 Transformer 模型解决长序列的运行时间和内存挑战。

许多近似注意力方法旨在降低注意力的计算和内存需求。这些方法从稀疏近似【引用说明：Kitaev et al., 2020 (Reformer) —使用局部敏感哈希近似注意力；Roy et al., 2021 —高效注意力的另一种稀疏方法。这些是减少注意力计算量的代表性工作】到低秩近似【引用说明：Wang et al., 2020 (Linformer) —通过低秩投影近似注意力；Katharopoulos et al., 2020 —线性注意力；Choromanski et al., 2020 (Performer) —使用随机特征近似 softmax。这些是从低秩角度简化注意力矩阵的工作】，以及它们的组合【引用说明：Beltagy et al., 2020 (Longformer) —结合局部和全局注意力；Zaheer et al., 2020 (BigBird) —结合随机、局部和全局注意力；Chen et al., 2021 (Scatterbrain) —统一稀疏和低秩注意力。这些工作尝试组合多种策略】。尽管这些方法将计算需求降低到线性或近线性，但许多方法没有展现出相对于标准注意力的实际墙钟时间加速，也没有获得广泛采用。一个主要原因是它们专注于减少 FLOPs（浮点运算次数，这可能与实际速度不相关），而忽略了内存访问（IO）的开销。

在本文中，我们论证了一个缺失的原则是使注意力算法具备 **IO 感知性**【引用说明：Aggarwal & Vitter, 1988 —关于排序和相关问题的 IO 复杂度的经典论文，是本文 IO 分析的理论基础】——即仔细考虑不同层级快速和慢速内存之间的读写（例如，快速的 GPU 片上 SRAM 与相对较慢的 GPU 高带宽内存 HBM 之间【引用说明：Jia et al., 2018 —剖析 GPU 内存层次结构和带宽，为本文提供了硬件参数】，如图 1 左）。在现代 GPU 上，计算速度已经超过了内存速度【引用说明：Nvidia, 2017/2020/2022 —Nvidia GPU 白皮书系列，说明计算与内存速度的差距在不断扩大】，Transformer 中的大多数操作是内存访问受限的【引用说明：Ivanov et al., 2021 —分析了深度学习操作的数据移动成本，说明 Transformer 中的瓶颈在于内存访问】。IO 感知算法在类似的内存受限操作中一直至关重要——当读写数据占据运行时间的很大部分时——例如数据库连接【引用说明：Ramakrishnan et al., 2003 —数据库系统教科书，IO 优化是经典主题】、图像处理【引用说明：Ragan-Kelley et al., 2013 (Halide) —图像处理领域的 IO 优化编译器，与本文目标类似】、数值线性代数【引用说明：Blackford et al., 2002 (BLAS) —基础线性代数子程序库，是 IO 优化的经典例子】等【引用说明：Williams et al., 2009 (Roofline 模型) —分析计算与内存带宽平衡的经典模型；Hennessy & Patterson, 2003 —计算机架构教科书】。然而，PyTorch 和 Tensorflow 等深度学习的常用 Python 接口不允许对内存访问进行细粒度控制。

我们提出 FlashAttention，一种以精确方式计算注意力、同时大幅减少内存访问的新注意力算法。我们的主要目标是避免将注意力矩阵读写到 HBM。这需要：(i) 在不访问整个输入的情况下计算 softmax 归约；(ii) 不为反向传播存储大型中间注意力矩阵。我们应用两种成熟的技术来解决这些挑战：(i) 我们重构注意力计算，将输入分成块，对输入块进行多次遍历，从而增量地执行 softmax 归约（也称为 **分块/tiling**）。(ii) 我们从前向传播中存储 softmax 归一化因子，以在反向传播中快速地在片上**重新计算**（recompute）注意力，这比从 HBM 读取中间注意力矩阵的标准方法更快。我们用 CUDA 实现 FlashAttention，以实现对内存访问的细粒度控制，并将所有注意力操作融合到一个 GPU 内核中。即使由于重计算导致 FLOPs 增加，我们的算法在**运行速度更快**（在 GPT-2 上最高 7.6 倍【引用说明：Radford et al., 2019 (GPT-2) —OpenAI 的 GPT-2 模型，本文使用它作为主要的实验基准之一】，图 1 右）的同时**使用更少内存**——关于序列长度是线性的——这得益于 HBM 访问量的大幅减少。

我们分析了 FlashAttention 的 IO 复杂度【引用说明：Aggarwal & Vitter, 1988 —再次引用，作为 IO 复杂度分析的理论框架】，证明它需要 $O(N^2d^2M^{-1})$ 次 HBM 访问（其中 d 是注意力头维度，M 是 SRAM 大小），而标准注意力需要 $\Omega(Nd+N^2)$ 次。对于 d 和 M 的典型值，

FlashAttention 比标准注意力少很多倍的 HBM 访问（最多少 9 倍）。此外，我们提供了一个下界，证明没有精确注意力算法能在所有 SRAM 大小上渐近改进 HBM 访问次数。
我们还展示 FlashAttention 可以作为一个有用的原语，通过克服近似注意力算法的内存访问开销问题来释放它们的潜力。作为概念验证，我们实现了块稀疏 FlashAttention，这是一种比 FlashAttention 还快 2–4 倍的稀疏注意力算法，可以扩展到 64K 的序列长度。我们证明块稀疏 FlashAttention 的 IO 复杂度比 FlashAttention 好一个与稀疏率成比例的因子。我们开源了 FlashAttention[a]。
我们通过实验验证了 FlashAttention 加速模型训练并通过建模更长上下文提高模型质量：

- **更快的模型训练**：FlashAttention 以墙钟时间更快地训练 Transformer 模型。BERT-large（序列长度 512）训练速度比 MLPerf 1.1 速度记录快 15%【引用说明：Mattson et al., 2020 (MLPerf) —机器学习训练性能基准，用于公平比较】；GPT-2（序列长度 1K）比 HuggingFace【引用说明：Wolf et al., 2020 —HuggingFace Transformers 库，广泛使用的基线实现】和 Megatron-LM【引用说明：Shoeybi et al., 2019 —Nvidia 的大模型训练框架，代表高度优化的基线】基线快 3 倍；长程竞技场（序列长度 1K–4K）快 2.4 倍。

- **更高质量的模型**：FlashAttention 将 Transformer 扩展到更长序列，从而提高质量。GPT-2 困惑度提升 0.7，长文档分类提升 6.4 个点【引用说明：Dai et al., 2022 —重新审视长文档分类中的 Transformer 模型，本文使用其实验设置】。FlashAttention 使得首个 Transformer 在 Path-X 挑战【引用说明：Tay et al., 2020 —再次引用 Long Range Arena，Path-X 是其中最难的任务】（序列长度 16K）上获得优于随机的表现成为可能。块稀疏 FlashAttention 使 Transformer 可扩展到更长序列（64K），首个在 Path-256 上取得优于随机表现的序列模型。

- **注意力基准测试**：FlashAttention 在常见序列长度（128 到 2K）上比标准注意力快最多 3 倍，可扩展到 64K。在序列长度不超过 512 时，FlashAttention 在速度和内存效率上都优于所有现有注意力方法；序列长度超过 1K 时，某些近似注意力方法（如 Linformer）开始变得更快。而块稀疏 FlashAttention 比我们所知的所有近似注意力方法都更快。

[a] https://github.com/HazyResearch/flash-attention

通俗解释

这一节首先告诉我们问题所在：Transformer 的核心——注意力机制——随着序列变长变得越来越慢，因为每个词都要和其他所有词「交流」。过去的解决方案都是「少算一些」（近似），但实际上并没有快多少。
为什么呢？因为真正的瓶颈不在于「算了多少次乘法」，而在于「数据在快慢内存之间搬运了多少次」。这就像你做饭时，切菜切得很快，但如果每切一刀都要跑去冰箱拿一次食材再放回去，那做饭的总时间还是很长。
FlashAttention 的核心思路是：把要切的菜分成小份，一次性从冰箱拿出来放在案板上（SRAM），全部切完再放回去，大幅减少「跑冰箱」（访问 HBM）的次数。具体使用了两个技巧：

1. **分块 (Tiling)**：把大矩阵切成小块，每次只处理一小块。

2. **重计算 (Recomputation)**：反向传播时不保存中间的大矩阵，而是重新算一遍——虽然多算了一些，但省去了大量的内存搬运，反而更快。

3 背景 (Background)

3.1 硬件性能 (Hardware Performance)

原文翻译

我们在此聚焦于 GPU。其他硬件加速器上的性能特征类似【引用说明：Jouppi et al., 2017 —Google TPU 论文；Jia et al., 2019 —更广泛的 AI 加速器剖析。说明类似的内存层次问题存在于各类加速器中】。

GPU 内存层次结构。GPU 内存层次结构（图 1 左）包含多种不同大小和速度的内存，较小的内存更快。以 A100 GPU 为例，它有 40–80GB 的高带宽内存（HBM），带宽为 1.5–2.0 TB/s，以及每个流式多处理器（SM）上 192KB 的片上 SRAM（共 108 个 SM），带宽估计约为 19 TB/s【引用说明：Jia et al., 2018, 2021 —详细剖析了 GPU 的内存层次和带宽，为本文提供具体的硬件数据】。片上 SRAM 比 HBM 快一个数量级，但在大小上小很多个数量级。随着计算速度相对于内存速度越来越快【引用说明：Nvidia GPU 白皮书系列—说明计算-内存速度差距持续扩大】，操作越来越受到内存（HBM）访问的瓶颈制约。因此利用快速 SRAM 变得更加重要。

执行模型。GPU 拥有大量线程来执行一个操作（称为内核/kernel）。每个内核从 HBM 加载输入到寄存器和 SRAM，进行计算，然后将输出写回 HBM。

性能特征。根据计算和内存访问的平衡，操作可以分为计算受限或内存受限。这通常用**算术强度**（arithmetic intensity）来衡量【引用说明：Williams et al., 2009 (Roofline 模型) —提出了用算术强度衡量操作瓶颈的经典模型】，即每字节内存访问的算术运算次数。

1. **计算受限**：操作时间由算术运算量决定，而内存访问时间远小于计算时间。典型例子包括内部维度较大的矩阵乘法和通道数较多的卷积。

2. **内存受限**：操作时间由内存访问次数决定，而计算时间远小于访问时间。例子包括大多数其他操作：逐元素操作（如激活函数、dropout）和归约操作（如求和、softmax、batch norm、layer norm）。

内核融合。加速内存受限操作的最常见方法是**内核融合**：如果有多个操作应用于同一输入，输入可以从 HBM 只加载一次，而不是每个操作各加载一次。编译器可以自动融合许多逐元素操作【引用说明：Li et al., 2020; Paszke et al., 2019 (PyTorch); Sabne et al., 2020 (XLA) —深度学习编译器的算子融合工作】。然而，在模型训练中，中间值仍需写入 HBM 以保存用于反向传播，这降低了简单内核融合的效果。

通俗解释

这一节解释了 GPU 的内存结构，可以类比为一个厨房：

- **HBM**（高带宽内存）= **冰箱**：容量大（40–80GB），但拿东西相对慢（1.5–2.0 TB/s）。
- **SRAM**（片上静态随机存储器）= **案板/砧板**：容量极小（每个处理单元仅 192KB），但速度极快（~19 TB/s），大约快 10 倍。

一个 GPU 操作（kernel）的过程就像一个菜谱步骤：从冰箱拿食材（从 HBM 读数据），在案板上处理（在 SRAM 中计算），然后放回冰箱（写回 HBM）。

操作分两类：**计算受限**的操作像是「切一大块肉」——切的时间长，跑冰箱的时间可以忽略；**内存受限**的操作像是「给食材撒盐」——撒的动作很快，但你一直在跑冰箱拿东西放东西，大部分时间花在了搬运上。注意力中的 softmax 等操作恰恰属于后者。

内核融合的思想就是：如果你要先撒盐、再撒胡椒、再撒孜然，不如一次性把三种调料拿到案板上一起撒，而不是跑三次冰箱。

数学详解：算术强度与瓶颈分类

算术强度（Arithmetic Intensity）定义为：

$$\text{算术强度} = \frac{\text{浮点运算次数 (FLOPs)}}{\text{内存访问字节数 (Bytes)}} \tag{1}$$

举例：假设一个 $N \times N$ 矩阵乘法 $C = A \times B$：

- FLOPs：$2N^2 \cdot N = 2N^3$（每个输出元素需要 N 次乘法和 $N-1$ 次加法）。
- 内存访问：读取 A 和 B 共 $2N^2$ 个元素，写入 C 共 N^2 个元素，总计 $3N^2$ 个元素，即 $3N^2 \times 4$ 字节（FP32）$= 12N^2$ 字节。
- 算术强度 $\approx \frac{2N^3}{12N^2} = \frac{N}{6}$。当 N 较大时，算术强度高，是**计算受限**的。

对比 softmax：输入 N 个数，需要求最大值、减去最大值、取指数、求和、除以总和：

- FLOPs：约 $5N$。
- 内存访问：读取 N 个元素 + 写入 N 个元素 = $2N$ 个元素 = $8N$ 字节。
- 算术强度 $\approx \frac{5N}{8N} \approx 0.625$。非常低，是**内存受限**的。

A100 GPU 的计算峰值约 312 TFLOPS (FP16)，HBM 带宽约 2 TB/s。Roofline 模型的「屋脊点」（ridge point）约 $\frac{312\times10^{12}}{2\times10^{12}} = 156$ FLOPs/Byte。算术强度低于 156 的操作都是内存受限的。softmax 的 0.625 远远低于此值。

3.2 标准注意力实现 (Standard Attention Implementation)

原文翻译

给定输入序列 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N\times d}$，其中 N 是序列长度，d 是注意力头维度，我们希望计算注意力输出 $\mathbf{O} \in \mathbb{R}^{N\times d}$：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N\times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N\times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N\times d}, \tag{2}$$

其中 softmax 按行应用。
标准注意力实现将矩阵 $\mathbf{S}$ 和 $\mathbf{P}$ **物化**（materialize）到 HBM，这需要 $O(N^2)$ 的内存。通常 $N \gg d$（例如，GPT-2 中 $N = 1024$，$d = 64$）。我们在算法 0 中描述了标准注意力实现。由于部分或大部分操作是内存受限的（如 softmax），大量的内存访问导致了较慢的墙钟时间。这个问题因为应用于注意力矩阵的其他逐元素操作而加剧，例如对 $\mathbf{S}$ 应用掩码或对 $\mathbf{P}$ 应用 dropout。因此，已有许多尝试融合多个逐元素操作，例如将掩码与 softmax 融合【引用说明：Shoeybi et al., 2019 (Megatron-LM) —在大模型训练框架中实现了 softmax 与 mask 的融合】。

算法 0：标准注意力实现

Algorithm 1 标准注意力实现 (Standard Attention Implementation)

Require: 矩阵 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N\times d}$，存储在 HBM 中。
1: 从 HBM 按块加载 $\mathbf{Q}, \mathbf{K}$，计算 $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$，将 $\mathbf{S}$ 写入 HBM。
2: 从 HBM 读取 $\mathbf{S}$，计算 $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$，将 $\mathbf{P}$ 写入 HBM。
3: 从 HBM 按块加载 $\mathbf{P}$ 和 $\mathbf{V}$，计算 $\mathbf{O} = \mathbf{P}\mathbf{V}$，将 $\mathbf{O}$ 写入 HBM。
4: 返回 $\mathbf{O}$。

算法 0 逐行解读

- **第 1 行**：把 **Q**（查询）和 **K**（键）从 HBM 读出来，做矩阵乘法得到 $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$（大小 $N \times N$），然后把这个巨大的矩阵**写回** HBM。这是第一次大规模 IO。
- **第 2 行**：再把 **S** 从 HBM **读出来**，按行做 softmax 得到 **P**，然后把 **P**（同样 $N \times N$）**写回** HBM。这是第二次大规模 IO。
- **第 3 行**：再把 **P** 和 **V** 从 HBM **读出来**，做矩阵乘法得到最终输出 **O**，写回 HBM。
- **核心问题**：$N \times N$ 的矩阵 **S** 和 **P** 来回搬运了多次——读 **S** 一次写一次，**P** 读一次写一次，总共对 N^2 大小的数据做了 $\Theta(N^2)$ 次内存访问。

通俗解释

标准注意力的计算过程分三步，每一步都要把大量数据搬进搬出 HBM（大内存）：

1. **第一步**：计算「谁和谁相似」——把 **Q**（查询）和 **K**（键）相乘，得到一个巨大的 $N \times N$ 相似度矩阵 **S**。如果 $N = 1024$，这个矩阵有 100 多万个元素！算完后要把整个矩阵写回 HBM。
2. **第二步**：对 **S** 做 softmax，转化为概率 **P**。需要从 HBM 读出 **S**，算完后把 **P** 写回 HBM。
3. **第三步**：用概率 **P** 对值 **V** 做加权求和。需要从 HBM 读出 **P** 和 **V**，算完后把结果 **O** 写回。

关键问题：**S** 和 **P** 这两个 $N \times N$ 的大矩阵需要反复搬进搬出 HBM，这就是瓶颈所在！

数学详解：标准注意力计算

符号说明：

- $\mathbf{Q} \in \mathbb{R}^{N \times d}$：查询矩阵（Query），$N$ 行（每个位置一行），d 列（注意力头维度）。
- $\mathbf{K} \in \mathbb{R}^{N \times d}$：键矩阵（Key），结构同 **Q**。
- $\mathbf{V} \in \mathbb{R}^{N \times d}$：值矩阵（Value），结构同 **Q**。
- $\mathbf{S} \in \mathbb{R}^{N \times N}$：注意力得分矩阵，$S_{ij} = q_i^\top k_j$，表示位置 i 对位置 j 的关注程度（未归一化）。
- $\mathbf{P} \in \mathbb{R}^{N \times N}$：注意力权重矩阵，$P_{ij} = \frac{e^{S_{ij}}}{\sum_{l=1}^{N} e^{S_{il}}}$，按行做 softmax，使每行和为 1。
- $\mathbf{O} \in \mathbb{R}^{N \times d}$：输出矩阵，$o_i = \sum_j P_{ij} v_j$，是值向量的加权和。

数值示例（$N = 3, d = 2$）：

设 $\mathbf{Q} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, \mathbf{K} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}, \mathbf{V} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}$。

第 1 步：$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$：

$$\mathbf{S} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 2 & 1 & 1 \end{pmatrix}$$

第 2 步：按行做 softmax，以第一行为例：

$$P_{1,:} = \text{softmax}([1, 0, 1]) = \frac{[e^1, e^0, e^1]}{e^1 + e^0 + e^1} = \frac{[2.718, 1, 2.718]}{6.436} = [0.422, 0.155, 0.422]$$

第 3 步：$\mathbf{O} = \mathbf{PV}$，以第一行为例：

$$o_1 = 0.422 \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} + 0.155 \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix} + 0.422 \cdot \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0.845 \\ 0.578 \end{pmatrix}$$

内存分析：存储 $\mathbf{S}$ 和 $\mathbf{P}$ 各需要 N^2 个元素。当 $N = 16384$ 时，仅 $\mathbf{S}$ 一个矩阵就需要 $16384^2 \times 2 = 512$ MB（FP16），这在 GPU 上是巨大的开销。

注意事项

1. **FLOPs $\neq$ 速度**：减少 FLOPs 不一定能加速。标准注意力的 FLOPs 为 $O(N^2 d)$，而 FlashAttention 的 FLOPs 实际上更多（因为重计算），但由于 HBM 访问减少，反而更快。
2. $N \gg d$ **是常见情况**：在 GPT-2 中 $N = 1024, d = 64$，所以 $N^2 = 1048576$ 远大于 $Nd = 65536$。这意味着 $N \times N$ 矩阵的存储和搬运成本远高于 $N \times d$ 矩阵。
3. **内存受限 vs 计算受限**：注意力中的 softmax 是内存受限操作。即使计算本身很快，频繁的 HBM 读写也会严重拖慢整体速度。

4 FlashAttention：算法、分析与扩展

4.1 使用分块与重计算的高效注意力算法

原文翻译

给定 HBM 中的输入 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$，我们的目标是计算注意力输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$ 并将其写入 HBM。我们的目标是将 HBM 访问量减少到亚二次（sub-quadratic in N）。

我们应用两种成熟的技术（分块、重计算）来克服在亚二次 HBM 访问中计算精确注意力的技术挑战。主要思想是将输入 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 分成块，从慢速 HBM 加载到快速 SRAM，然后计算关于这些块的注意力输出。通过在合并之前用正确的归一化因子缩放每个块的输出，我们在最后得到正确的结果。

分块 (Tiling)。我们按块计算注意力。Softmax 耦合了 $\mathbf{K}$ 的列，所以我们用缩放分解大 softmax【引用说明：Milakov & Gimelshein, 2018 —在线 softmax 算法，可以增量计算 softmax 而无需一次看到所有输入；Kitaev et al., 2020 (Reformer); Rabe & Staats, 2021 —这些工作也利用了类似的分块 softmax 技巧】。为数值稳定性，向量 $x \in \mathbb{R}^B$ 的 softmax 按如下方式计算：

$$\begin{aligned} m(x) &:= \max_i \; x_i, \\ f(x) &:= \begin{bmatrix} e^{x_1 - m(x)} & \cdots & e^{x_B - m(x)} \end{bmatrix}, \\ \ell(x) &:= \sum_i f(x)_i, \\ \text{softmax}(x) &:= \frac{f(x)}{\ell(x)}. \end{aligned} \tag{3}$$

对于向量 $x^{(1)}, x^{(2)} \in \mathbb{R}^B$，我们可以将拼接向量 $x = [x^{(1)} \; x^{(2)}] \in \mathbb{R}^{2B}$ 的 softmax 分解为：

$$\begin{aligned} m(x) &= \max(m(x^{(1)}), m(x^{(2)})), \\ f(x) &= \left[e^{m(x^{(1)}) - m(x)} f(x^{(1)}) \quad e^{m(x^{(2)}) - m(x)} f(x^{(2)}) \right], \\ \ell(x) &= e^{m(x^{(1)}) - m(x)} \ell(x^{(1)}) + e^{m(x^{(2)}) - m(x)} \ell(x^{(2)}), \quad \text{softmax}(x) = \frac{f(x)}{\ell(x)}. \end{aligned} \tag{4}$$

因此，如果我们跟踪一些额外的统计量（$m(x), \ell(x)$），就可以逐块计算 softmax。
重计算 (Recomputation)。我们的一个目标是不存储 $O(N^2)$ 的中间值用于反向传播。反向传播通常需要矩阵 $\mathbf{S}, \mathbf{P} \in \mathbb{R}^{N \times N}$ 来计算关于 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 的梯度。然而，通过存储输出 $\mathbf{O}$ 和 softmax 归一化统计量 (m, ℓ)，我们可以在反向传播中从 SRAM 中的 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 块轻松重新计算注意力矩阵 $\mathbf{S}$ 和 $\mathbf{P}$。这可以看作是一种选择性梯度检查点【引用说明：Griewank & Walther, 2008 —梯度检查点的经典参考；Chen et al., 2016 —将梯度检查点应用于深度学习训练以节省内存】。虽然梯度检查点已被建议用于减少最大内存需求【引用说明：Rabe & Staats, 2021 —使用梯度检查点实现内存高效注意力】，但所有已知的实现都必须以速度换取内存。相比之下，即使有更多 FLOPs，我们的重计算由于减少了 HBM 访问，**加速**了反向传播。
实现细节：内核融合。分块使我们能够在一个 CUDA 内核中实现算法：从 HBM 加载输入，执行所有计算步骤（矩阵乘法、softmax、可选的掩码和 dropout、矩阵乘法），然后将结果写回 HBM。这避免了将输入和输出反复读写于 HBM。

通俗解释

分块 (Tiling) 的直觉：
假设你要给全班 100 个学生的考试成绩做「归一化」（softmax），使得所有成绩之和为 1。标准做法需要一次看到所有 100 个分数。但 FlashAttention 的做法是：

1. 先看前 25 个同学的分数，记下最高分和总分。
2. 再看下 25 个同学，更新最高分和总分。
3. 如此反复，最后得到和一次性看完所有人完全相同的结果。

关键在于：每次只需要带上两个「备忘录」——当前最高分 m 和归一化因子 ℓ。
重计算 (Recomputation) 的直觉：
训练神经网络需要「反向传播」来更新参数。反向传播通常需要前向传播中的中间结果。标准做法是把中间的 $N \times N$ 大矩阵保存在 HBM 中。但 FlashAttention 说："我不存了，反向传播时我重新算一遍！"虽然多算了一些乘法，但省下了大量的 HBM 读写。由于注意力是内存受限的操作，省下的搬运时间远超多算的计算时间。

数学详解：分块 Softmax 推导

问题：如何在不一次看到所有数据的情况下，正确计算 softmax?
数值稳定的 softmax：直接计算 e^{x_i} 可能溢出（如 e^{1000}），所以先减去最大值：

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}} = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}, \quad m = \max_j x_j$$

分块计算：假设 $x = [x^{(1)}, x^{(2)}]$，即两块拼在一起。
第一块处理后我们得到：$m^{(1)} = \max(x^{(1)})$，$f_i^{(1)} = e^{x_i^{(1)} - m^{(1)}}$，$\ell^{(1)} = \sum_i f_i^{(1)}$。
处理第二块时：$m^{(2)} = \max(x^{(2)})$，$f_i^{(2)} = e^{x_i^{(2)} - m^{(2)}}$，$\ell^{(2)} = \sum_i f_i^{(2)}$。
合并：新的全局最大值 $m = \max(m^{(1)}, m^{(2)})$。需要修正第一块的结果：

$$f_i^{(1)}(\text{修正后}) \quad = e^{x_i^{(1)} - m} = e^{x_i^{(1)} - m^{(1)}} \cdot e^{m^{(1)} - m} = f_i^{(1)} \cdot e^{m^{(1)} - m}$$

类似地修正 ℓ：

$$\ell = e^{m^{(1)} - m} \ell^{(1)} + e^{m^{(2)} - m} \ell^{(2)}$$

数值例子：$x = [3, 1, 4, 1, 5, 9]$，分成 $x^{(1)} = [3, 1, 4]$ 和 $x^{(2)} = [1, 5, 9]$。
处理 $x^{(1)}$：

- $m^{(1)} = 4$

- $f^{(1)} = [e^{3-4}, e^{1-4}, e^{4-4}] = [e^{-1}, e^{-3}, e^{0}] = [0.368, 0.050, 1.000]$
- $\ell^{(1)} = 0.368 + 0.050 + 1.000 = 1.418$

处理 $x^{(2)}$:

- $m^{(2)} = 9$
- $f^{(2)} = [e^{1-9}, e^{5-9}, e^{9-9}] = [e^{-8}, e^{-4}, e^{0}] = [0.000335, 0.0183, 1.000]$
- $\ell^{(2)} = 0.000335 + 0.0183 + 1.000 = 1.0187$

合并:

- $m = \max(4, 9) = 9$
- 修正 $\ell^{(1)}$：$e^{4-9} \cdot 1.418 = e^{-5} \cdot 1.418 = 0.00674 \cdot 1.418 = 0.00956$
- $\ell = 0.00956 + e^{9-9} \cdot 1.0187 = 0.00956 + 1.0187 = 1.0282$

可以验证，$\text{softmax}([3, 1, 4, 1, 5, 9])$ 的归一化常数确实是 $\sum e^{x_i - 9} = e^{-6} + e^{-8} + e^{-5} + e^{-8} + e^{-4} + 1 \approx 1.0282$。结果完全正确！

4.2 FlashAttention 算法伪代码

原文翻译——算法 1：FlashAttention

输入：矩阵 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$（在 HBM 中），片上 SRAM 大小为 M。

Algorithm 2 FlashAttention（简化版前向传播）

Require: 矩阵 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$（在 HBM 中），片上 SRAM 大小 M。
1: 设置块大小 $B_c = \lceil M/(4d) \rceil$，$B_r = \min(\lceil M/(4d) \rceil, d)$。
2: 初始化 $\mathbf{O} = (0)_{N \times d}$，$\ell = (0)_N$，$m = (-\infty)_N$（在 HBM 中）。
3: 将 $\mathbf{Q}$ 分成 $T_r = \lceil N/B_r \rceil$ 个块 $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$（每块 $B_r \times d$），将 $\mathbf{K}, \mathbf{V}$ 分成 $T_c = \lceil N/B_c \rceil$ 个块（每块 $B_c \times d$）。
4: 将 $\mathbf{O}$ 分成 T_r 块，ℓ 分成 T_r 块，m 分成 T_r 块。
5: **for** $1 \le j \le T_c$ **do**
6: 　将 $\mathbf{K}_j, \mathbf{V}_j$ 从 HBM 加载到片上 SRAM。
7: 　**for** $1 \le i \le T_r$ **do**
8: 　　将 $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ 从 HBM 加载到片上 SRAM。
9: 　　片上计算 $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^\top \in \mathbb{R}^{B_r \times B_c}$。
10: 　　片上计算 $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$，$\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$，$\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$。
11: 　　片上计算 $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij})$，$\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij}$。
12: 　　将 $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} \big(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j\big)$ 写入 HBM。
13: 　　将 $\ell_i \leftarrow \ell_i^{\text{new}}$，$m_i \leftarrow m_i^{\text{new}}$ 写入 HBM。
14: 　**end for**
15: **end for**
16: 返回 $\mathbf{O}$。

定理 1（正确性）：算法 1 返回 $\mathbf{O} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$，FLOPs 为 $O(N^2 d)$，额外内存（除输入输出外）为 $O(N)$。

算法 1 逐行解读

- **第 1 行——块大小设定**：$B_c = \lceil M/(4d) \rceil$ 确保 $\mathbf{K}_j$（$B_c \times d$）和 $\mathbf{V}_j$（$B_c \times d$）能放进 SRAM。除以 $4d$ 是因为还要给 $\mathbf{Q}_i$、$\mathbf{O}_i$ 和 $\mathbf{S}_{ij}$ 留空间。B_r 还受 d 的限制，确保 $\mathbf{S}_{ij}$（$B_r \times B_c$）也能放进 SRAM。
- **第 2 行——初始化**：$\mathbf{O}$ 全零（最终输出），ℓ 全零（softmax 归一化分母的累积），m 全负无穷（softmax 数值稳定性的最大值追踪）。
- **第 3–4 行——分块**：将所有矩阵切成小块。
- **第 5 行——外循环**：遍历 $\mathbf{K}, \mathbf{V}$ 的每一块。
- **第 6 行——加载 $\mathbf{K}_j, \mathbf{V}_j$**：把当前的 key/value 块从 HBM 搬到 SRAM（快速内存）。在整个内循环期间它们不动。
- **第 7 行——内循环**：对固定的 $\mathbf{K}_j, \mathbf{V}_j$，遍历 $\mathbf{Q}$ 的所有块。
- **第 8 行——加载 $\mathbf{Q}_i$ 等**：把当前 query 块和已有统计量搬到 SRAM。
- **第 9 行——矩阵乘法**：在 SRAM 上计算小块的注意力得分 $\mathbf{S}_{ij}$，大小仅 $B_r \times B_c$，远小于 $N \times N$。
- **第 10 行——局部 softmax 统计**：对 $\mathbf{S}_{ij}$ 的每行求最大值、做指数、求行和——这是当前块的局部 softmax 统计量。
- **第 11 行——合并统计量**：把当前块的统计量与之前所有块的累积统计量合并。$e^{m_i - m_i^{\text{new}}}$ 是修正因子，因为全局最大值可能发生了变化。
- **第 12 行——增量更新输出**：用合并后的统计量修正之前的 $\mathbf{O}_i$ 并加上当前块的贡献。这是整个算法最核心的一步！
- **第 13 行——写回统计量**：更新 HBM 中的 ℓ_i 和 m_i。
- **关键洞察**：整个过程中，$N \times N$ 的注意力矩阵永远没有被完整地存储或写入 HBM。只有大小为 $B_r \times B_c$ 的小块短暂存在于 SRAM 中。

数学详解：输出增量更新公式推导

算法第 (e) 步的更新公式看起来很复杂，我们来拆解它。
目标：在处理完第 j 块 $\mathbf{K}$ 后，$\mathbf{O}_i$ 应该等于对前 j 块做 softmax 注意力的结果。
设已经处理了前 $j-1$ 块，当前状态为：

- m_i：之前所有块中 $\mathbf{S}$ 第 i 行的最大值
- ℓ_i：之前所有块的修正指数和
- $\mathbf{O}_i$：之前的部分注意力输出（已乘以 $1/\ell_i$ 归一化）

处理第 j 块后，新的统计量为：

- $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij})$
- 之前的 ℓ_i 需要修正：$\ell_i \to e^{m_i - m_i^{\text{new}}} \ell_i$（因为最大值可能变了）
- 新块的贡献：$e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij}$
- $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij}$

输出更新：

$$\mathbf{O}_i^{\text{new}} = \frac{1}{\ell_i^{\text{new}}}\Big(\underbrace{e^{m_i - m_i^{\text{new}}} \ell_i \cdot \mathbf{O}_i}_{\text{之前的贡献（修正后）}} + \underbrace{e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j}_{\text{当前块的新贡献}} \Big) \tag{5}$$

这里 $\ell_i \cdot \mathbf{O}_i$ 是之前的「未归一化累计和」（因为 $\mathbf{O}_i$ 在上一步已经被 $1/\ell_i^{\text{old}}$ 归一化过，乘以 ℓ_i 就恢复成未归一化形式），$e^{m_i - m_i^{\text{new}}}$ 是因为全局最大值发生了变化而需要的修正因子。
简单数值示例（$B_r = 1, B_c = 2, d = 1$）：
设 $q_1 = 1$，$k = [2, 3, 1]$（三个键，分两块 $k^{(1)} = [2, 3]$ 和 $k^{(2)} = [1]$），$v = [10, 20, 5]$。
初始：$m_1 = -\infty, \ell_1 = 0, O_1 = 0$。
第 1 块（$k^{(1)} = [2, 3], v^{(1)} = [10, 20]$）：

- $S = q_1 \cdot k^{(1)} = [1 \cdot 2, 1 \cdot 3] = [2, 3]$
- $\tilde{m} = 3, \tilde{P} = [e^{2-3}, e^{3-3}] = [0.368, 1], \tilde{\ell} = 1.368$
- $m_1^{\text{new}} = \max(-\infty, 3) = 3$
- $\ell_1^{\text{new}} = e^{-\infty-3} \cdot 0 + e^{3-3} \cdot 1.368 = 1.368$
- $O_1 = \frac{1}{1.368}(0 + 1 \cdot [0.368 \cdot 10 + 1 \cdot 20]) = \frac{23.68}{1.368} = 17.31$

第 2 块（$k^{(2)} = [1], v^{(2)} = [5]$）：

- $S = q_1 \cdot k^{(2)} = [1]$
- $\tilde{m} = 1, \tilde{P} = [e^{1-1}] = [1], \tilde{\ell} = 1$
- $m_1^{\text{new}} = \max(3, 1) = 3$
- $\ell_1^{\text{new}} = e^{3-3} \cdot 1.368 + e^{1-3} \cdot 1 = 1.368 + 0.135 = 1.503$
- $O_1 = \frac{1}{1.503}(1.368 \cdot e^{3-3} \cdot 17.31 + e^{1-3} \cdot 1 \cdot 5) = \frac{1.368 \cdot 17.31 + 0.135 \cdot 5}{1.503} = \frac{24.36}{1.503} = 16.21$

验证：标准 softmax attention：$S = [2, 3, 1]$, $\text{softmax}([2, 3, 1]) = \frac{[e^2, e^3, e^1]}{e^2 + e^3 + e^1} = \frac{[7.389, 20.086, 2.718]}{30.193} = [0.245, 0.665, 0.090]$，$O_1 = 0.245 \times 10 + 0.665 \times 20 + 0.090 \times 5 = 16.21$。**结果一致！**

4.3 分析：FlashAttention 的 IO 复杂度

原文翻译

我们分析 FlashAttention 的 IO 复杂度，展示与标准注意力相比 HBM 访问的显著减少。我们还提供了一个下界，证明没有精确注意力算法能在所有 SRAM 大小上渐近改进 HBM 访问次数。证明在附录中。
定理 2：设 N 为序列长度，d 为注意力头维度，M 为 SRAM 大小，满足 $d \le M \le Nd$。标准注意力（算法 0）需要 $\Theta(Nd + N^2)$ 次 HBM 访问，而 FlashAttention（算法 1）需要 $\Theta(N^2 d^2 M^{-1})$ 次 HBM 访问。
对于 d（64–128）和 M（约 100KB）的典型值，d^2 远小于 M，因此 FlashAttention 比标准实现少很多倍的 HBM 访问。这带来了更快的执行速度和更小的内存占用。
证明的主要思想：给定 SRAM 大小为 M，我们可以加载大小为 $\Theta(M)$ 的 $\mathbf{K}, \mathbf{V}$ 块（算法 1 第 6 行）。对于每个 $\mathbf{K}$ 和 $\mathbf{V}$ 块，我们遍历所有 $\mathbf{Q}$ 块（算法 1 第 8 行）以计算中间值，导致对 $\mathbf{Q}$ 进行 $\Theta(NdM^{-1})$ 次遍历。每次遍历加载 $\Theta(Nd)$ 个元素，总计 $\Theta(N^2 d^2 M^{-1})$ 次 HBM 访问。
命题 3（下界）：设条件同上。不存在一种算法能以 $o(N^2 d^2 M^{-1})$ 次 HBM 访问在所有

$M \in [d, Nd]$ 范围内计算精确注意力。
证明依赖于这样的事实：当 $M = \Theta(Nd)$ 时，任何算法都必须执行 $\Omega(N^2d^2M^{-1}) = \Omega(Nd)$ 次 HBM 访问。这种在 M 的子范围上的下界在流式算法文献中很常见【引用说明：Woodruff, 2004 —关于流式算法最优复杂度的工作】。
我们验证了 HBM 访问次数是注意力运行时间的主要决定因素。在图 2（左）中，虽然 FlashAttention 的 FLOP 数更高（由于反向传播中的重计算），但 HBM 访问少得多，导致运行时间快得多。

注意力方法	标准	FlashAttention
GFLOPs	66.6	75.2
HBM 读/写 (GB)	40.3	4.4
运行时间 (ms)	41.7	7.3

（GPT-2 medium 配置：序列长度 1024，头维度 64，16 个头，批量大小 64，A100 GPU。）

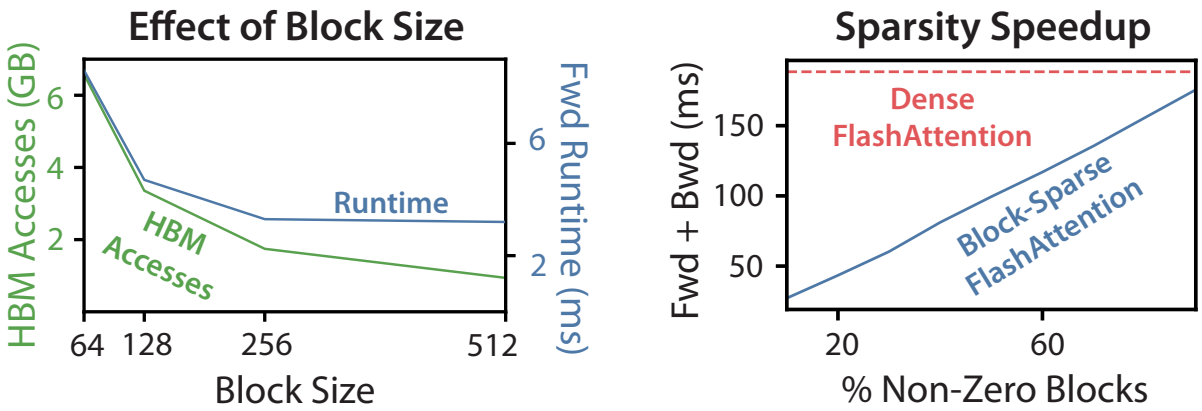

Figure 2: **图 2（论文原图）。左：** GPT-2 medium 在 A100 GPU 上的前向 + 反向传播运行时间（序列长度 1024，头维度 64，16 头，批量大小 64）。HBM 访问是影响运行时间的主要因素。**中：** FlashAttention 前向传播运行时间（相同配置），不同块大小 B_c 对应不同数量的 HBM 访问。更少的 HBM 访问 = 更快的运行时间，直到受限于其他因素。**右：** 块稀疏 FlashAttention 的运行时间（序列长度 4K）与稀疏度成比例地减少。

图 2 解读

这张三面板图是论文中最有说服力的实验证据之一：
左面板（表格）——FLOPs vs HBM 访问 vs 运行时间：

- 标准注意力：66.6 GFLOPs 计算量，40.3 GB 的 HBM 读写量，运行时间 41.7 ms。
- FlashAttention：75.2 GFLOPs 计算量（**比标准多 13%**！），但 HBM 读写仅 4.4 GB（**减少约 9 倍**），运行时间 7.3 ms（**快 5.7 倍**）。
- **核心洞察**：虽然 FlashAttention 做的算术运算更多（因为反向传播中的重计算），但由于 HBM 访问大幅减少，实际速度反而快得多。这有力地证明了 **IO（内存访问）而非 FLOPs 才是瓶颈**。

中面板——块大小对运行时间的影响：

- 横轴是块大小 B_c，越大意味着每次加载更多数据到 SRAM，遍历次数越少。
- 纵轴是前向传播运行时间。

- 可以看到：随着 B_c 增大，HBM 访问减少，运行时间几乎线性下降——直到 B_c 超过约 256 后趋于平缓（此时瓶颈从内存转为计算，或者块已经太大无法放入 SRAM）。
- 这完美验证了定理 2 的理论预测：IO 复杂度 $\propto 1/B_c \propto 1/M$。

右面板——稀疏度与运行时间：

- 横轴是稀疏度比例（非零块的比例），越小意味着跳过越多计算。
- 纵轴是运行时间。
- 块稀疏 FlashAttention 的运行时间与稀疏度几乎成正比——验证了命题 4。

通俗解释

这一节用数学严格证明了 FlashAttention 确实更快。
标准注意力需要的 HBM 搬运量：$\Theta(Nd + N^2)$。当 N 很大时，N^2 项主导——搬运的数据量和序列长度的平方成正比。
FlashAttention 需要的 HBM 搬运量：$\Theta(N^2d^2/M)$。这里 M 是 SRAM 大小（约 100KB）。关键在于分母多了一个 M/d^2，即：

$$\frac{\text{标准注意力的 HBM 访问}}{\text{FlashAttention 的 HBM 访问}} \approx \frac{N^2}{N^2d^2/M} = \frac{M}{d^2}$$

代入典型值：$M \approx 100\text{KB} = 100{,}000\,\text{bytes} \approx 50{,}000$ 个 FP16 数，$d = 64$，所以 $M/d^2 \approx 50000/4096 \approx 12$。也就是说 FlashAttention 的 HBM 访问大约少 12 倍！
下界的意义：论文还证明了 FlashAttention 基本上已经是最优的——不可能有其他精确注意力算法在所有 SRAM 大小下比它还快。
实测数据完美地验证了理论：FlashAttention 的 FLOPs（75.2 GFLOPs）比标准注意力（66.6 GFLOPs）多 13%（因为重计算），但 HBM 访问（4.4 GB）只有标准注意力（40.3 GB）的约 1/9，运行时间（7.3 ms）只有标准注意力（41.7 ms）的约 1/6。

数学详解：IO 复杂度证明梗概

标准注意力的 IO 复杂度：
步骤 1：计算 $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$：读 $\mathbf{Q}, \mathbf{K}$（$2Nd$ 元素），写 $\mathbf{S}$（N^2 元素）$\Rightarrow \Theta(Nd + N^2)$。
步骤 2：$\mathbf{P} = \text{softmax}(\mathbf{S})$：读 $\mathbf{S}$，写 $\mathbf{P} \Rightarrow \Theta(N^2)$。
步骤 3：$\mathbf{O} = \mathbf{P}\mathbf{V}$：读 $\mathbf{P}, \mathbf{V}$，写 $\mathbf{O} \Rightarrow \Theta(N^2 + Nd)$。
总计：$\Theta(Nd + N^2)$。
FlashAttention 的 IO 复杂度：
块大小约束：$\mathbf{K}_j$ 和 $\mathbf{V}_j$ 各 $B_c \times d$，$\mathbf{Q}_i$ 和 $\mathbf{O}_i$ 各 $B_r \times d$，$\mathbf{S}_{ij}$ 为 $B_r \times B_c$，都需要放入 SRAM：

$$2B_cd + 2B_rd + B_rB_c = O(M)$$

选取 $B_c = \Theta(M/d)$，$B_r = \Theta(\min(M/d, d))$。
块数：$T_c = N/B_c = \Theta(Nd/M)$。
外循环遍历 T_c 个 $(\mathbf{K}_j, \mathbf{V}_j)$ 块，每次加载 $2B_cd = \Theta(M)$ 元素。
内循环对每个 j 遍历所有 T_r 个 $\mathbf{Q}_i$ 块，每次读写 $\Theta(B_rd)$ 元素。
每个内循环共读写 $T_r \cdot B_rd = \Theta(Nd)$ 元素。外循环执行 $T_c = \Theta(Nd/M)$ 次。
总 HBM 访问 $= \Theta(Nd) + T_c \cdot \Theta(Nd) = \Theta\left(\frac{N^2d^2}{M}\right)$。
数值代入：$N = 1024, d = 64, M = 100\text{KB} = 50000$(FP16 数)：

- 标准注意力：$Nd + N^2 = 65536 + 1048576 \approx 1.1 \times 10^6$
- FlashAttention：$N^2d^2/M = 1048576 \times 4096/50000 \approx 8.6 \times 10^4$

- 比值：≈ 13 倍 ✓

4.4 扩展：块稀疏 FlashAttention

原文翻译

我们将 FlashAttention 扩展到近似注意力：我们提出块稀疏 FlashAttention，其 IO 复杂度比 FlashAttention 小一个与稀疏度成比例的因子。
给定输入 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ 和一个掩码矩阵 $\tilde{\mathbf{M}} \in \{0,1\}^{N \times N}$，我们要计算：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S} \odot \mathbb{1}_{\tilde{\mathbf{M}}}), \quad \mathbf{O} = \mathbf{P}\mathbf{V}, \tag{6}$$

其中 $(\mathbf{S} \odot \mathbb{1}_{\tilde{\mathbf{M}}})_{kl} = S_{kl}$（若 $\tilde{M}_{kl} = 1$）或 $-\infty$（若 $\tilde{M}_{kl} = 0$）。我们要求 $\tilde{\mathbf{M}}$ 具有块形式。
给定预定义的块稀疏掩码 $\mathbf{M} \in \{0,1\}^{N/B_r \times N/B_c}$，我们只需在算法中跳过零块即可。
命题 4：块稀疏 FlashAttention 需要 $\Theta(Nd + N^2 d^2 M^{-1} s)$ 次 HBM 访问，其中 s 是块稀疏掩码中非零块的比例。
应用块稀疏可以直接按稀疏度比例改善 IO 复杂度中的主要项。对于长序列 N，s 通常设为 $N^{-1/2}$【引用说明：Child et al., 2019 (Sparse Transformer) —提出了固定稀疏模式的长序列 Transformer】 或 $N^{-1}\log N$【引用说明：Zaheer et al., 2020 (BigBird); Beltagy et al., 2020 (Longformer); Dao et al., 2022a (Pixelated Butterfly) —这些工作使用了不同的稀疏模式】，导致 $\Theta(N\sqrt{N})$ 或 $\Theta(N \log N)$ 的 IO 复杂度。对于下游实验，我们使用固定的蝶形稀疏模式【引用说明：Dao et al., 2022a (Pixelated Butterfly) —提出了蝶形稀疏模式，被证明可以近似任意稀疏模式；Dao et al., 2020 (Kaleidoscope) —证明了蝶形矩阵的表达能力】。

通俗解释

标准的 FlashAttention 中，每个查询都要和所有的键进行交互。块稀疏 FlashAttention 则允许跳过某些块——比如，你知道距离很远的词之间通常没什么关系，就可以直接跳过它们之间的计算。
实现方式非常简单：在算法的外循环和内循环中，当遇到掩码中为 0 的块时，直接跳过（不从 HBM 加载，也不计算）。如果只有 s 比例的块是非零的，那么 IO 次数就按比例减少。
这就像老师改作业时，如果知道某些学生肯定满分（或者某些题不用改），就直接跳过，改作业的总时间自然就少了。

5 实验 (Experiments)

5.1 使用 FlashAttention 的更快模型

原文翻译

BERT。FlashAttention 实现了我们所知的最快的单节点 BERT 训练速度。我们在 Wikipedia 上训练 BERT-large 模型【引用说明：Devlin et al., 2019 —提出了 BERT 预训练模型】。与 Nvidia 提交给 MLPerf 1.1【引用说明：Mattson et al., 2020 —MLPerf 基准】 的训练速度记录相比，我们的实现快 15%。

BERT 实现	训练时间（分钟）
Nvidia MLPerf 1.1	20.0 ± 1.5
FlashAttention（本文）	$\mathbf{17.4 \pm 1.4}$

（在 8× A100 GPU 上训练到 72.0% 掩码语言模型准确率，10 次运行平均。）

GPT-2。FlashAttention 在大规模 OpenWebText 数据集【引用说明：Gokaslan & Clancy, 2019 —OpenWebText 数据集，GPT-2 的开源复现训练集】 上训练 GPT-2【引用说明：Radford et al., 2019 —GPT-2 模型】 比广泛使用的 HuggingFace【引用说明：Wolf et al., 2020 —Transformers 库】 和 Megatron-LM【引用说明：Shoeybi et al., 2019 —Nvidia 大模型训练框架】 实现更快。

模型实现	困惑度	训练时间（加速比）
GPT-2 small – HuggingFace	18.2	9.5 天 (1.0×)
GPT-2 small – Megatron-LM	18.2	4.7 天 (2.0×)
GPT-2 small – FlashAttention	18.2	**2.7 天 (3.5×)**
GPT-2 medium – HuggingFace	14.2	21.0 天 (1.0×)
GPT-2 medium – Megatron-LM	14.3	11.5 天 (1.8×)
GPT-2 medium – FlashAttention	14.3	**6.9 天 (3.0×)**

（8× A100 GPU，序列长度 1K。FlashAttention 达到与基线相同的困惑度。）

长程竞技场。我们在 LRA 基准【引用说明：Tay et al., 2020 —长程竞技场基准】 上比较标准 Transformer（使用标准注意力或 FlashAttention）。FlashAttention 比标准注意力快 2.4 倍。块稀疏 FlashAttention 比所有测试过的近似注意力方法都更快。

模型	ListOps	Text	Retrieval	Image	Path.	平均	加速
Transformer	36.0	63.6	81.6	42.3	72.7	59.3	–
FlashAttention	37.6	63.9	81.4	43.5	72.7	59.8	2.4×
块稀疏 FlashAttention	37.0	63.0	81.3	43.6	73.3	59.6	**2.8**×
Linformer	35.6	55.9	77.7	37.8	67.6	54.9	2.5×
Linear Attn	38.8	63.2	80.7	42.6	72.5	59.6	2.3×
Performer	36.8	63.6	82.2	42.1	69.9	58.9	1.8×
Reformer	36.5	63.8	78.5	39.6	69.4	57.6	1.3×

通俗解释

实验结果表明 FlashAttention 在三个主要基准上都实现了显著加速：

1. **BERT**：业界训练速度记录原来是 20 分钟，FlashAttention 只要 17.4 分钟，快了 15%。
2. **GPT-2**：比最流行的 HuggingFace 实现快 3–3.5 倍！GPT-2 small 原本要 9.5 天训练，现在只要 2.7 天。
3. **LRA**：在所有五个长序列任务上，FlashAttention 在保持几乎相同准确率的同时，快 2.4 倍。近似注意力方法虽然有类似加速，但准确率通常更差。

特别值得注意的是：FlashAttention 不改变模型本身（不近似），所以困惑度和标准实现完全一样。它只是让同一个模型训练得更快。

5.2 更长序列带来更好模型

原文翻译

长上下文语言建模。FlashAttention 的运行时和内存效率使我们可以将 GPT-2 的上下文长度增加 4 倍，同时仍比 Megatron-LM 的优化实现更快。使用 FlashAttention 和上下文长度 4K 的 GPT-2 比使用上下文长度 1K 的 Megatron GPT-2 仍快 30%，同时困惑度好 0.7。

模型实现	上下文长度	困惑度	训练时间
GPT-2 small – Megatron-LM	1K	18.2	4.7 天 (1.0×)
GPT-2 small – FlashAttention	1K	18.2	**2.7 天** (1.7×)
GPT-2 small – FlashAttention	2K	17.6	3.0 天 (1.6×)
GPT-2 small – FlashAttention	4K	**17.5**	3.6 天 (1.3×)

长文档分类。使用 FlashAttention 训练更长序列的 Transformer 在 MIMIC-III【引用说明：Johnson et al., 2016 —重症监护病房病人出院总结数据集】 和 ECtHR【引用说明：Chalkidis et al., 2019, 2021 —欧洲人权法院案例数据集】 数据集上提高了表现。

	512	1024	2048	4096	8192	16384
MIMIC-III	52.8	50.7	51.7	54.6	56.4	**57.1**
ECtHR	72.2	74.3	77.1	78.6	**80.7**	79.2

序列长度 16K 在 MIMIC 上比长度 512 高 4.3 个点；序列长度 8K 在 ECtHR 上比长度 512 高 8.5 个点。

Path-X 和 Path-256。这是 LRA 基准中最具挑战性的任务，旨在测试长上下文能力。任务是判断 128×128（或 256×256）黑白图像中两个点之间是否有路径连接，图像以逐像素方式输入 Transformer。此前，所有 Transformer 模型要么耗尽内存，要么只能达到随机水平的表现【引用说明：Tay et al., 2020 —报告了此前所有模型在 Path-X 上均失败】。一直有人在寻找能处理如此长上下文的替代架构【引用说明：Gu et al., 2022 (S4) —状态空间模型，是处理长序列的替代架构】。

我们首次展示 Transformer 能解决 Path-X 和 Path-256：FlashAttention 在 Path-X 上达到 61.4% 准确率；块稀疏 FlashAttention 使 Transformer 扩展到序列长度 64K，在 Path-256 上达到 63.1% 准确率。

模型	Path-X	Path-256
Transformer	✗	✗
Linformer	✗	✗
Performer	✗	✗
Reformer	✗	✗
FlashAttention	**61.4**	✗
块稀疏 FlashAttention	56.0	**63.1**

（✗ 表示未能达到优于随机的表现或耗尽内存。）

通俗解释

这些实验展示了 FlashAttention 最令人兴奋的成果——让 Transformer "看得更远"：

1. **更长上下文 = 更好的语言模型**：使用 4K 上下文长度的 GPT-2 比 1K 上下文的 GPT-2 困惑度好了 0.7（从 18.2 降到 17.5），而训练时间仍然比 Megatron 使用 1K 上下文时更短！这就像给学生更多的上下文阅读材料，自然能写出更好的文章。

2. **长文档分类**：对于需要阅读长文档的任务（如医疗记录分类），更长的序列 = 更高的准确率。MIMIC-III 数据集的平均文档长度约 2400 词，当模型能看到 16K 个词时，准确率从 52.8% 提升到 57.1%。

3. **Path-X/Path-256 的突破**：这是一个标志性的成就。Path-X 需要处理 $128 \times 128 = 16384$ 个像素的序列。之前没有任何 Transformer 模型能做到这一点——它们要么内存不够，要么准确率和扔硬币一样。FlashAttention 首次让 Transformer 在这个任务上取得有意义的表现（61.4%，而随机是 50%）。

5.3 注意力基准测试

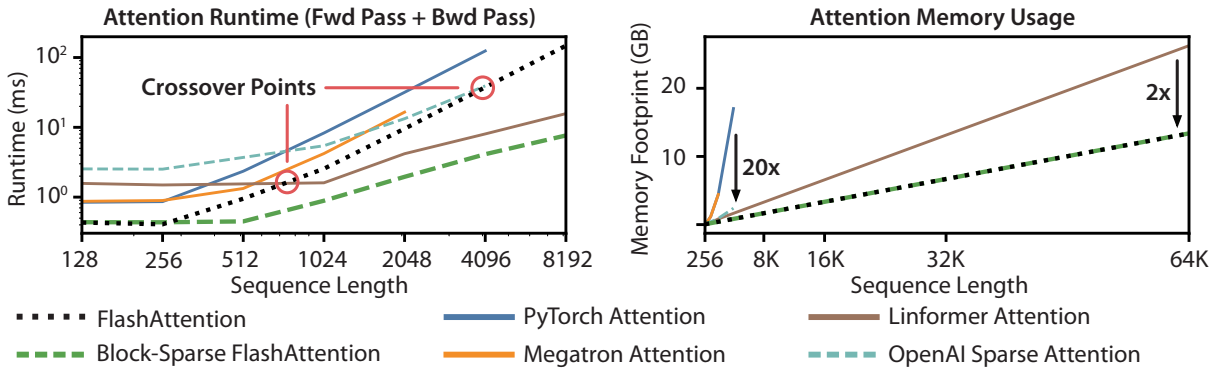

Figure 3: **图 3（论文原图）。左：**前向 + 反向传播运行时间。**右：**注意力内存使用量。

原文翻译

我们在一块 A100 GPU（40 GB HBM）上变化序列长度，测量 FlashAttention 和块稀疏 FlashAttention 相对于各种注意力基线的运行时间和内存使用，带有 dropout 和 padding 掩码。

运行时间。FlashAttention 的运行时间随序列长度二次增长，但显著快于**精确注意力**基线，比 PyTorch 实现快最多 3 倍。许多近似/稀疏注意力机制的运行时间随序列长度线性增长，但 FlashAttention 在短序列上仍然更快（因为更少的内存访问）。近似注意力的运行时间在序列长度 512–1024 之间开始与 FlashAttention 交叉。块稀疏 FlashAttention 在所有序列长度上都比所有已知的精确、稀疏和近似注意力实现更快。

内存占用。FlashAttention 和块稀疏 FlashAttention 的内存占用相同，随序列长度线性增长。FlashAttention 比**精确注意力**基线节省最多 20 倍内存，比**近似注意力**基线也更节省内存。除 Linformer 外，所有其他算法在 A100 GPU 上在 64K 之前就耗尽了内存，而 FlashAttention 仍比 Linformer 节省 2 倍。

图 3 解读

这张图是论文中最直观的性能对比图，分左右两部分。

左图——运行时间 vs 序列长度：

- 横轴为序列长度（从 128 到 65536），纵轴为运行时间（对数刻度，ms）。
- **红/橙色线**（标准注意力/PyTorch/Megatron）：运行时间随 N 二次增长，曲线陡峭上升。
- **蓝色线**（FlashAttention）：同样是二次增长，但曲线**明显更低**——在所有常见序列长度上快 2–3 倍。
- **绿色/紫色线**（近似注意力方法如 Linformer、Performer 等）：运行时间线性增长（斜率更小），但在短序列（≤ 1024）上反而比 FlashAttention 慢，因为它们的常数因子更大（IO 开销更高）。
- **深蓝色线**（块稀疏 FlashAttention）：在**所有**序列长度上都最快——比精确注意力快得多，也比近似注意力快。
- 关键观察：大约在序列长度 1024–2048 处，Linformer 等近似方法开始与 FlashAttention 速度持平甚至更快。但块稀疏 FlashAttention 始终保持领先。

右图——内存占用 vs 序列长度：

- **红/橙色线**（标准注意力）：内存随 N^2 增长，在 $N > 16K$ 时就超过 40GB GPU 内存而 OOM（Out of Memory）。
- **蓝色线**（FlashAttention）：内存随 N **线性**增长！这是因为不存储 $N \times N$ 矩阵。即使在 $N = 64K$ 时仍然可以运行。
- FlashAttention 比标准注意力节省最多 **20 倍**内存，比大多数近似方法也更节省。

实验核心总结

1. **速度**：FlashAttention 在所有常见序列长度上比标准注意力快 2–3 倍；块稀疏版本最快。
2. **内存**：$O(N)$ 而非 $O(N^2)$，节省高达 20 倍。
3. **质量**：精确计算，不丢失精度；更长的上下文带来更好的模型质量。
4. **里程碑**：首次在 Path-X（16K 长度）和 Path-256（64K 长度）上取得有意义的 Transformer 表现。

6 局限与未来方向 (Limitations and Future Directions)

原文翻译

编译到 CUDA。我们当前构建 IO 感知注意力实现的方法需要为每个新的注意力实现编写一个新的 CUDA 内核。这要求用比 PyTorch 低得多的语言编写注意力算法，需要大量的工程努力。实现也可能无法在不同 GPU 架构之间迁移。这些局限性表明需要一种方法，支持用高级语言（如 PyTorch）编写注意力算法，并编译为 IO 感知的 CUDA 实现——类似于图像处理领域的 Halide 所做的努力【引用说明：Ragan-Kelley et al., 2013 (Halide) —图像处理领域的调度-算法分离编译器，是期望中的方向】。
IO 感知的深度学习。我们相信 IO 感知方法可以扩展到注意力之外。注意力是 Transformer 中最耗内存的计算，但深度网络中的每一层都会接触 GPU HBM。我们希望这项工作能启发其他模块的 IO 感知实现。
多 GPU IO 感知方法。我们的 IO 感知注意力实现在单 GPU 上是常数因子内最优的。然而，注意力计算可以在多个 GPU 间并行化【引用说明：Recht et al., 2013 —关于大规模并行化的工作】。使用多 GPU 增加了 IO 分析的另一层——需要考虑 GPU 间的数据传输。

通俗解释

作者诚实地指出了 FlashAttention 的三个局限：

1. **编程门槛高**：目前每种新的注意力变体都需要手写 CUDA 代码，这就像每次想做一道新菜都要自己造一口新锅。理想情况是有一个编译器自动把高级代码转换为高效的 CUDA 内核。
2. **不只是注意力**：FlashAttention 的 IO 优化思想可以应用到深度学习的其他部分（如 MLP 层、归一化层等），但目前只优化了注意力。
3. **多 GPU**：当前只优化了单 GPU 内的内存层次。多 GPU 训练时，GPU 之间的数据传输又是一个新的优化层面。

7 附录 A：相关工作 (Related Work)

原文翻译

IO 感知运行时优化。优化快速/慢速内存之间读写的广泛概念在计算机科学中有着悠久的历史，并以许多名称为人所知。我们在这项工作中与 IO 复杂度分析文献建立了最直接的联系【引用说明：Aggarwal & Vitter, 1988】，但内存层次结构的概念是基础性的，已经以许多形式出现：从工作集模型【引用说明：Denning, 1968 —程序行为的工作集模型】到数据局部性【引用说明：Wolf & Lam, 1991】，到算术强度的 Roofline 模型【引用说明：Williams et al., 2009】，到可扩展性分析【引用说明：McSherry et al., 2015】，到计算机架构的标准教科书【引用说明：Hennessy & Patterson, 2003】。

结构化矩阵的高效 ML 模型。矩阵乘法是大多数 ML 模型的核心计算瓶颈。为降低计算复杂度，已有许多方法学习更高效的矩阵集合——**结构化矩阵**，它具有亚二次的参数数量和运行时间。常见例子包括稀疏和低秩矩阵，以及信号处理中的快速变换（傅里叶变换等）。我们用于块稀疏注意力的蝶形模式源自蝶形矩阵【引用说明：Dao et al., 2019/2020 —蝶形矩阵的学习和表达能力相关工作】及其乘积已被证明能以几乎最优的运行时间和参数数量表达任何结构化矩阵。

稀疏训练。我们的块稀疏 FlashAttention 可以看作是使稀疏模型训练更高效的一步。稀疏模型在推理时的模型压缩（剪枝）中取得了成功。彩票假说【引用说明：Frankle & Carlin, 2018/2019/2020 —提出了彩票假说，认为大网络中存在与原网络一样好的小子网络】表明大型密集网络中存在表现与原始密集网络一样好的小型子网络。

高效 Transformer。基于 Transformer 的模型已成为 NLP【引用说明：Devlin et al., 2019 —BERT】和计算机视觉【引用说明：Dosovitskiy et al., 2020 (ViT) —Vision Transformer】中最广泛使用的架构。克服注意力二次瓶颈的方法包括：基于哈希（稀疏）的近似如 Reformer【引用说明：Kitaev et al., 2020】和 Smyrf【引用说明：Daras et al., 2020】；低秩近似如 Performer【引用说明：Choromanski et al., 2020】；以及组合方法如 Longformer【引用说明：Beltagy et al., 2020】、BigBird【引用说明：Zaheer et al., 2020】、Scatterbrain【引用说明：Chen et al., 2021】等。

还有几条使用非注意力模块来建模更长上下文的工作线。HiPPO【引用说明：Gu et al., 2020】及其扩展，特别是 S4【引用说明：Gu et al., 2022 —状态空间模型，通过将历史投影到多项式基上实现高效的长序列建模】，将历史投影到多项式基上，结合了 CNN（高效训练）、RNN（高效推理）和连续模型（对采样率变化的鲁棒性）的优势。

8 附录 B：算法细节

8.1 内存高效的前向传播

原文翻译

使注意力内存高效的主要挑战是 softmax 耦合了 $\mathbf{K}$（和 $\mathbf{V}$）的列。我们的方法是分别计算 softmax 归一化常数以解耦列。

定义归一化常数：

$$L_i = \sum_j e^{q_i^\top k_j} \tag{7}$$

设 v_j 为 $\mathbf{V}$ 的第 j 列（行），输出的第 i 行为：

$$o_i = \sum_j P_{ij} v_j = \sum_j \frac{e^{q_i^\top k_j}}{L_i} v_j \tag{8}$$

一旦计算了 L_i，就可以通过反复累加 $\frac{e^{q_i^\top k_j}}{L_i} v_j$ 来计算 o_i，不需要额外内存。因此前向传播可

用 $O(N)$ 额外内存完成。

8.2 FlashAttention 完整前向传播（算法 2）

原文翻译——完整前向传播

完整的注意力前向传播包括 softmax 缩放 τ（通常为 $1/\sqrt{d}$）、掩码函数 mask（如 padding mask 或因果 mask）和 dropout：

$$\mathbf{S} = \tau\mathbf{Q}\mathbf{K}^\top, \quad \mathbf{S}^{\text{masked}} = mask(\mathbf{S}), \quad \mathbf{P} = \text{softmax}(\mathbf{S}^{\text{masked}}),$$
$$\mathbf{P}^{\text{dropped}} = \text{dropout}(\mathbf{P}, p_{\text{drop}}), \quad \mathbf{O} = \mathbf{P}^{\text{dropped}}\mathbf{V}$$

Algorithm 3 FlashAttention 完整前向传播

Require: $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N\times d}$（HBM），SRAM 大小 M，缩放常数 τ，掩码函数 mask，dropout 概率 p_{drop}。

1: 初始化伪随机数生成器状态 $\mathcal{R}$，保存到 HBM。
2: 设置块大小 $B_c = \lceil M/(4d)\rceil$，$B_r = \min(\lceil M/(4d)\rceil, d)$。
3: 初始化 $\mathbf{O} = (0)_{N\times d}$，$\ell = (0)_N$，$m = (-\infty)_N$（HBM）。
4: 将 $\mathbf{Q}$ 分成 T_r 块，$\mathbf{K}, \mathbf{V}$ 分成 T_c 块；$\mathbf{O}, \ell, m$ 同样分块。
5: **for** $1 \le j \le T_c$ **do**
6: 　将 $\mathbf{K}_j, \mathbf{V}_j$ 从 HBM 加载到 SRAM。
7: 　**for** $1 \le i \le T_r$ **do**
8: 　　将 $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ 从 HBM 加载到 SRAM。
9: 　　片上计算 $\mathbf{S}_{ij} = \tau\mathbf{Q}_i\mathbf{K}_j^\top \in \mathbb{R}^{B_r\times B_c}$。
10: 　　片上计算 $\mathbf{S}_{ij}^{\text{masked}} = mask(\mathbf{S}_{ij})$。
11: 　　片上计算 $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}^{\text{masked}})$，$\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij}^{\text{masked}} - \tilde{m}_{ij})$，$\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij})$。
12: 　　片上计算 $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij})$，$\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}}\ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}}\tilde{\ell}_{ij}$。
13: 　　片上计算 $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$。
14: 　　写入 $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}\big(\text{diag}(\ell_i)e^{m_i - m_i^{\text{new}}}\mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}}\tilde{\mathbf{P}}_{ij}^{\text{dropped}}\mathbf{V}_j\big)$ 到 HBM。
15: 　　写入 $\ell_i \leftarrow \ell_i^{\text{new}}$，$m_i \leftarrow m_i^{\text{new}}$ 到 HBM。
16: 　**end for**
17: **end for**
18: 返回 $\mathbf{O}, \ell, m, \mathcal{R}$。

算法 2 vs 算法 1 的区别

算法 2 是算法 1 的完整版，新增了三个实际训练中必不可少的步骤：

- **第 1 行——保存随机种子** $\mathcal{R}$：Dropout 需要随机掩码，但如果保存整个 $N \times N$ 的掩码需要 $O(N^2)$ 内存。所以只保存随机数生成器的**状态**，反向传播时用同样的种子重新生成。
- **第 9 行——缩放** τ：标准注意力会除以 $\sqrt{d}$，即 $\tau = 1/\sqrt{d}$，防止内积值过大导致 softmax 梯度消失。
- **第 10 行——掩码**：将不需要注意的位置设为 $-\infty$（如 padding token 或因果掩码中的未来位置），使 softmax 后这些位置的权重为 0。
- **第 13 行——Dropout**：对注意力权重随机置零，防止过拟合。注意 dropout 也在 SRAM 中完成，不需要额外的 HBM 访问。

- **第 17 行——额外返回值**：保存 $\ell, m, \mathcal{R}$ 供反向传播使用（而非存储 $N \times N$ 的 **S** 或 **P**!）。

8.3 FlashAttention 反向传播

原文翻译——标准注意力反向传播

Algorithm 4 标准注意力反向传播

Require: $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{dO} \in \mathbb{R}^{N \times d}$，$\mathbf{P} \in \mathbb{R}^{N \times N}$（HBM）。

1: 加载 $\mathbf{P}, \mathbf{dO}$，计算 $\mathbf{dV} = \mathbf{P}^\top \mathbf{dO}$，写入 HBM。
2: 加载 $\mathbf{dO}, \mathbf{V}$，计算 $\mathbf{dP} = \mathbf{dO}\,\mathbf{V}^\top \in \mathbb{R}^{N \times N}$，写入 HBM。
3: 读取 $\mathbf{P}, \mathbf{dP}$，计算 $dS_{ij} = P_{ij}(dP_{ij} - \sum_l P_{il}\, dP_{il})$，写入 HBM。
4: 加载 $\mathbf{dS}, \mathbf{K}$，计算 $\mathbf{dQ} = \mathbf{dS}\,\mathbf{K}$，写入 HBM。
5: 加载 $\mathbf{dS}, \mathbf{Q}$，计算 $\mathbf{dK} = \mathbf{dS}^\top \mathbf{Q}$，写入 HBM。
6: 返回 $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$。

算法 3（标准反向）逐行解读

标准反向传播的问题更严重：

- 它需要 $\mathbf{P} \in \mathbb{R}^{N \times N}$（前向传播中保存的注意力权重矩阵），占用 $O(N^2)$ 内存。
- 第 2 行又计算出 $\mathbf{dP} \in \mathbb{R}^{N \times N}$，第 3 行计算 $\mathbf{dS} \in \mathbb{R}^{N \times N}$——三个 $N \times N$ 大矩阵来回搬运。
- 总 HBM 访问量为 $\Theta(Nd + N^2)$，与前向传播一样是二次的。

Algorithm 5 FlashAttention 完整反向传播

Require: $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$，$\ell, m \in \mathbb{R}^N$（HBM），SRAM 大小 M，缩放 τ，掩码 mask，dropout p_{drop}，随机状态 $\mathcal{R}$。

1: 设置随机数生成器状态为 $\mathcal{R}$。
2: 设置块大小 B_c, B_r（同前向）。
3: 分块 $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO}, \ell, m$（同前向）。
4: 初始化 $\mathbf{dQ} = (0)_{N \times d}$，$\mathbf{dK} = (0)_{N \times d}$，$\mathbf{dV} = (0)_{N \times d}$（HBM），并分块。
5: **for** $1 \le j \le T_c$ **do**
6: 　加载 $\mathbf{K}_j, \mathbf{V}_j$ 到 SRAM。
7: 　初始化 $\tilde{\mathbf{dK}}_j = (0)_{B_c \times d}$，$\tilde{\mathbf{dV}}_j = (0)_{B_c \times d}$（SRAM）。
8: 　**for** $1 \le i \le T_r$ **do**
9: 　　加载 $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, \ell_i, m_i$ 到 SRAM。
10: 　　片上计算 $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^\top$。
11: 　　片上计算 $\mathbf{S}_{ij}^{\text{masked}} = mask(\mathbf{S}_{ij})$。
12: 　　片上计算 $\mathbf{P}_{ij} = \text{diag}(\ell_i)^{-1} \exp(\mathbf{S}_{ij}^{\text{masked}} - m_i)$。
13: 　　片上生成 dropout 掩码 $\mathbf{Z}_{ij}$（用 $\mathcal{R}$ 恢复与前向相同的掩码）。
14: 　　片上计算 $\mathbf{P}_{ij}^{\text{dropped}} = \mathbf{P}_{ij} \circ \mathbf{Z}_{ij}$。
15: 　　片上计算 $\tilde{\mathbf{dV}}_j \leftarrow \tilde{\mathbf{dV}}_j + (\mathbf{P}_{ij}^{\text{dropped}})^\top \mathbf{dO}_i$。
16: 　　片上计算 $\mathbf{dP}_{ij}^{\text{dropped}} = \mathbf{dO}_i \mathbf{V}_j^\top$。
17: 　　片上计算 $\mathbf{dP}_{ij} = \mathbf{dP}_{ij}^{\text{dropped}} \circ \mathbf{Z}_{ij}$。
18: 　　片上计算 $D_i = \text{rowsum}(\mathbf{dO}_i \circ \mathbf{O}_i) \in \mathbb{R}^{B_r}$。
19: 　　片上计算 $\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - D_i)$。
20: 　　写入 $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \tau\, \mathbf{dS}_{ij}\, \mathbf{K}_j$ 到 HBM。
21: 　　片上计算 $\tilde{\mathbf{dK}}_j \leftarrow \tilde{\mathbf{dK}}_j + \tau\, \mathbf{dS}_{ij}^\top\, \mathbf{Q}_i$。
22: 　**end for**
23: 　写入 $\mathbf{dK}_j \leftarrow \tilde{\mathbf{dK}}_j$，$\mathbf{dV}_j \leftarrow \tilde{\mathbf{dV}}_j$ 到 HBM。
24: **end for**
25: 返回 $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$。

算法 4（FlashAttention 反向）逐行解读——核心创新

这是 FlashAttention 最精妙的部分。与标准反向传播相比，有三个关键创新：

创新 1——重新计算 S 和 P（第 10–12 行）：

- 标准方法在前向传播时把 $\mathbf{P} \in \mathbb{R}^{N \times N}$ 存到 HBM，反向时读出来。
- FlashAttention 不存 $\mathbf{P}$，而是利用前向保存的 ℓ, m 和原始的 $\mathbf{Q}, \mathbf{K}$，在 SRAM 中**重新计算**出 $\mathbf{P}_{ij}$。$\mathbf{P}_{ij} = \text{diag}(\ell_i)^{-1} \exp(\mathbf{S}_{ij}^{\text{masked}} - m_i)$ 之所以成立，是因为 ℓ_i 和 m_i 是前向传播最终的统计量。
- 虽然多算了一次 $\mathbf{Q}_i \mathbf{K}_j^\top$（增加 FLOPs），但省去了从 HBM 读写 $N \times N$ 矩阵的巨大开销。

创新 2——重新生成 dropout 掩码（第 13–14 行）：

- 不保存 $N \times N$ 的 dropout 掩码，而是用前向传播时保存的随机种子 $\mathcal{R}$ 重新生成**完全相同**的掩码。
- 这把额外内存从 $O(N^2)$ 降到了 $O(1)$（只需一个种子值）。

创新 3——$D_i = \text{rowsum}(\mathbf{dO}_i \circ \mathbf{O}_i)$ 代替 $P_{i:}^\top dP_{i:}$（第 18 行）：

- 数学上 $D_i = do_i^\top o_i$，只需要两个 d 维向量的点积。

- 而原本的 $P_{i:}^\top dP_{i:}$ 需要两个 N 维向量——在 N 很大时根本放不进 SRAM。
- 这个化简是解析推导的结果（见附录 B 的反向传播推导部分）。

dK 的累积策略（第 7, 21, 22 行）：

- $\mathbf{dK}_j$ 在 SRAM 中用 $\tilde{\mathbf{dK}}_j$ 累积，内循环结束后一次性写回 HBM，减少写入次数。
- 而 $\mathbf{dQ}_i$ 因为在内循环中每次 j 都要更新，无法在 SRAM 中完全累积，所以每次直接加到 HBM 中的 $\mathbf{dQ}_i$（第 20 行）。

原文翻译

假设有标量损失函数 ϕ，输出梯度为 $\mathbf{dO}$（即 $\frac{\partial \phi}{\partial \mathbf{O}}$），我们要计算输入梯度 $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$。
$\mathbf{V}$ 的梯度最简单：$\mathbf{dV} = \mathbf{P}^\top \mathbf{dO}$，即：

$$dv_j = \sum_i P_{ij}\, do_i = \sum_i \frac{e^{q_i^\top k_j}}{L_i}\, do_i \tag{9}$$

对于 $\mathbf{Q}$ 和 $\mathbf{K}$ 的梯度，我们需要经过 $\mathbf{dP}$ 和 $\mathbf{dS}$。
$\mathbf{dP} = \mathbf{dO}\,\mathbf{V}^\top$，故 $dP_{ij} = do_i^\top v_j$。
softmax 的 Jacobian 为 $\mathrm{diag}(y) - yy^\top$（其中 $y = \mathrm{softmax}(x)$），因此：

$$dS_{i:} = P_{i:} \circ dP_{i:} - (P_{i:}^\top dP_{i:})\, P_{i:} \tag{10}$$

定义：

$$D_i = P_{i:}^\top dP_{i:} = \sum_j \frac{e^{q_i^\top k_j}}{L_i}\, do_i^\top v_j = do_i^\top o_i \tag{11}$$

最终的 $\mathbf{Q}$ 和 $\mathbf{K}$ 梯度为：

$$dq_i = \sum_j \frac{e^{q_i^\top k_j}}{L_i} (do_i^\top v_j - D_i)\, k_j \tag{12}$$

$$dk_j = \sum_i \frac{e^{q_i^\top k_j}}{L_i} (do_i^\top v_j - D_i)\, q_i \tag{13}$$

因此反向传播也可用 $O(N)$ 额外内存完成。

数学详解：反向传播梯度推导

为什么 $D_i = do_i^\top o_i$?
这是一个精巧的化简。回忆 $D_i = P_{i:}^\top dP_{i:} = \sum_j P_{ij} dP_{ij}$。
其中 $dP_{ij} = do_i^\top v_j$，$P_{ij} = \frac{e^{q_i^\top k_j}}{L_i}$。代入：

$$D_i = \sum_j \frac{e^{q_i^\top k_j}}{L_i}\, do_i^\top v_j = do_i^\top \sum_j \frac{e^{q_i^\top k_j}}{L_i} v_j = do_i^\top o_i$$

最后一步用到了前向传播的定义 $o_i = \sum_j P_{ij} v_j$。
这个化简很关键：D_i 的原始定义需要对 N 个元素求和（P_{ij} 和 dP_{ij} 都是长 N 的向量），而 $do_i^\top o_i$ 只是两个长 d 的向量的点积！在 FlashAttention 中，我们已经存了 $\mathbf{O}$，所以可以直接用 $do_i^\top o_i$ 计算 D_i，而不需要把整个 $\mathbf{P}$ 矩阵存下来。

softmax Jacobian 推导:
设 $y = \text{softmax}(x)$，则 $y_i = \frac{e^{x_i}}{\sum_k e^{x_k}}$。
对 x_j 求导:

$$\frac{\partial y_i}{\partial x_j} = \begin{cases} y_i(1-y_j) & \text{if } i = j \\ -y_i y_j & \text{if } i \neq j \end{cases} = y_i(\delta_{ij} - y_j) = [\text{diag}(y) - yy^\top]_{ij}$$

所以 $\frac{\partial y}{\partial x} = \text{diag}(y) - yy^\top$。

9 附录 C：正确性与复杂度证明

原文翻译——定理 1 的证明（正确性）

通过对外循环迭代次数 j 进行归纳。
设 $\mathbf{K}_{:j}$ 为 $\mathbf{K}$ 的前 jB_c 行，$\mathbf{S}_{:,:j} = \mathbf{Q}\mathbf{K}_{:j}^\top$，$\mathbf{P}_{:,:j} = \text{softmax}(\mathbf{S}_{:,:j})$。设 $m^{(j)}, \ell^{(j)}, \mathbf{O}^{(j)}$ 为第 j 次外循环迭代后 HBM 中的值。
我们要证明：在第 j 次外循环迭代后，

$$m^{(j)} = \text{rowmax}(\mathbf{S}_{:,:j}), \quad \ell^{(j)} = \text{rowsum}(\exp(\mathbf{S}_{:,:j} - m^{(j)})), \quad \mathbf{O}^{(j)} = \mathbf{P}_{:,:j}\,\mathbf{V}_{:j}$$

基例（$j = 0$）：初始化 $m = -\infty, \ell = 0, \mathbf{O} = 0$，对应空输入，成立。
归纳步骤：假设对 j 成立，证明对 $j+1$ 成立。更新后：

$$\begin{aligned} \mathbf{O}^{(j+1)} &= \text{diag}(\ell^{(j+1)})^{-1}\big(\text{diag}(\ell^{(j)})e^{m^{(j)}-m^{(j+1)}}\mathbf{O}^{(j)} + e^{\tilde{m}-m^{(j+1)}}\exp(\mathbf{S}_{j:j+1} - \tilde{m})\mathbf{V}_{j:j+1}\big) \\ &= \text{diag}(\ell^{(j+1)})^{-1}\big(e^{-m^{(j+1)}}\exp(\mathbf{S}_{:,:j})\mathbf{V}_{:j} + e^{-m^{(j+1)}}\exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}\big) \\ &= \text{softmax}(\mathbf{S}_{:,:j+1})\mathbf{V}_{:j+1} \end{aligned} \tag{14}$$

当 $j = T_c$ 时，得到 $\mathbf{O} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$。

原文翻译——下界证明（命题 3）

反证法。假设存在一个算法，在所有 $M \in [d, Nd]$ 上以 $o(N^2d^2/M)$ 次 HBM 访问计算精确注意力。
取 $M = \Theta(Nd)$，则 HBM 访问次数为 $o(N^2d^2/(Nd)) = o(Nd)$。
但注意力的输入（$\mathbf{Q}, \mathbf{K}, \mathbf{V}$）和输出 $\mathbf{O}$ 总共有 $\Theta(Nd)$ 个元素，它们从 HBM 开始，因此任何算法至少需要 $\Omega(Nd)$ 次 HBM 访问。矛盾！

通俗解释

下界的证明非常优雅简洁：当 SRAM 足够大（$M = \Theta(Nd)$），可以把所有输入都放进去，此时 FlashAttention 的复杂度变成 $\Theta(Nd)$——而这恰好是读取输入的最小成本。所以不可能更快了。
这就像说：不管你搬运技术多好，从冰箱拿出食材这个动作本身就是最低成本——你不可能比 “把所有东西拿出来一次” 更快。

10 附录 D：扩展细节

10.1 块稀疏 FlashAttention（算法 5）

Algorithm 6 块稀疏 FlashAttention 前向传播

Require: $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$（HBM），SRAM 大小 M，缩放 τ，掩码 mask，dropout p_{drop}，块稀疏掩码 $\mathbf{M} \in \{0,1\}^{N/B_r \times N/B_c}$。

1: 初始化随机数生成器状态 $\mathcal{R}$，保存到 HBM。
2: 初始化 $\mathbf{O} = (0)_{N \times d}$，$\ell = (0)_N$，$m = (-\infty)_N$（HBM）。
3: 分块 $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \ell, m$（同算法 2）。
4: **for** $1 \le j \le T_c$ **do**
5: 加载 $\mathbf{K}_j, \mathbf{V}_j$ 到 SRAM。
6: **for** $1 \le i \le T_r$ **do**
7: **if** $M_{ij} \neq 0$ **then**
8: 加载 $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ 到 SRAM。
9: 片上计算 $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^\top$。
10: 片上计算 $\mathbf{S}_{ij}^{\text{masked}} = mask(\mathbf{S}_{ij})$。
11: 片上计算 $\tilde{m}_{ij}, \tilde{\mathbf{P}}_{ij}, \tilde{\ell}_{ij}$（同算法 2）。
12: 片上更新 $m_i^{\text{new}}, \ell_i^{\text{new}}$（同算法 2）。
13: 片上计算 $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$。
14: 写入更新后的 $\mathbf{O}_i, \ell_i, m_i$ 到 HBM（同算法 2）。
15: **end if**
16: **end for**
17: **end for**
18: 返回 $\mathbf{O}, \ell, m, \mathcal{R}$。

算法 5 逐行解读

块稀疏 FlashAttention 与完整 FlashAttention（算法 2）**几乎完全相同**，唯一的区别在**第 7 行**：

- 增加了一个判断 `if` $M_{ij} \neq 0$——如果块稀疏掩码 $\mathbf{M}$ 的 (i,j) 位置为 0，意味着查询块 i 不需要关注键块 j，**直接跳过**。
- 不加载 $\mathbf{Q}_i$，不计算 $\mathbf{S}_{ij}$，不做 softmax——完全不消耗 HBM 带宽或计算资源。
- 如果只有比例 s 的块是非零的，那么内循环的执行次数减少为原来的 s 倍。

直觉类比：如果算法 1 是让每个学生看每块信息板，那么块稀疏版本就是根据课程表，只让相关学科的学生看相应的信息板——化学系的同学不需要去看文学的信息板。
稀疏模式选择：论文中使用了蝶形稀疏模式（butterfly pattern），它能用较少的非零块近似任意稀疏结构，同时具有良好的硬件友好性。

原文翻译

与 Rabe & Staats (2021) 的比较。
两种方法在概念上都对注意力矩阵的块进行操作，使用分块/softmax 缩放技术。两种方法都避免在前向传播中存储大注意力矩阵，并在反向传播中重计算它。
第一个主要区别：Rabe & Staats 专注于减少总内存占用（最大 GPU 内存需求），而 FlashAttention 专注于减少内存访问次数（读写次数）。内存访问量是运行时间的主要决定因素。结果是 FlashAttention 比标准注意力快 2–4 倍，而 Rabe & Staats 与标准注意力速度相当或略慢。

第二个区别：信息在块之间传递的方式不同。Rabe & Staats 为每个块保存一个临时输出副本，最后才合并；FlashAttention 增量更新输出，只需一个输出副本。
第三个区别：反向传播计算方式不同。Rabe & Staats 使用梯度检查点重计算注意力矩阵和每块的临时输出；FlashAttention 通过解析推导简化了反向传播，只重计算注意力矩阵，不重计算临时输出。
潜在扩展方向：

- **多 GPU 注意力**：在同一节点的多个 GPU 间协同计算注意力，考虑不同内存层次的不对称性。
- **稀疏 MLP 层**：许多稀疏 MLP 层实际是内存受限的，IO 感知实现可以释放稀疏性的潜力。
- **核方法机器学习**：注意力矩阵是低秩矩阵 $\mathbf{QK}^\top$ 的函数。类似地，核矩阵的每个元素是两个低维数据点的函数。KeOps 库【引用说明：Feydy et al., 2020; Charlier et al., 2021 —通过减少内存读写加速核操作的库】是减少内存读写加速核操作的成功例子。

11 附录 E：完整实验结果

原文翻译

BERT 训练细节：使用 LAMB 优化器，学习率 3.75e-3，批量大小 448，最多训练 7100 步。使用 FP16 精度（Apex AMP O2）。在 8× A100-80GB GPU 上训练，每次训练 16–19 分钟，平均 10 次结果。
GPT-2 训练细节：有效批量大小 512，使用梯度累积。AdamW 优化器，GPT-2 small 学习率 6e-4，GPT-2 medium 学习率 1.5e-4，权重衰减 0.1。所有模型训练 400K 步，使用混合精度训练。使用 OpenWebText 数据集和 GPT-2 BPE 分词器。在 8× A100-40GB GPU 上训练。
与 Apex FMHA 的比较：

注意力方法	128	256	512
Apex FMHA 前向	0.10	0.29	1.14
FlashAttention 前向	**0.08**	**0.22**	**0.81**
Apex FMHA 反向	**0.17**	**0.52**	**1.81**
FlashAttention 反向	0.20	0.53	2.00
Apex FMHA 总计	**0.27**	0.81	2.95
FlashAttention 总计	0.28	**0.75**	**2.81**

（单位：毫秒。A100 GPU，批量大小 64，16 头，头维度 64。）
FlashAttention 前向传播更快（不需存储注意力矩阵），反向传播略慢（需重计算），总体在序列长度 ≥ 256 时更快。

图 4 解读

这张图是 FlashAttention "精确性" 的直接证据。图中展示了训练过程中验证集困惑度（perplexity）随步数的变化：

- **蓝色线**：HuggingFace 标准实现训练的 GPT-2。
- **橙色线**：使用 FlashAttention 训练的 GPT-2。
- 两条曲线**几乎完全重合**——说明 FlashAttention 计算的注意力结果与标准实现**数值上一致**。

这点至关重要：FlashAttention 不是近似方法，不会损失模型质量。它用完全不同的计算顺序和内存访问模式，得到的是**数学上完全相同**的结果。唯一的区别是速度更快（相同步数下 FlashAttention 用的墙钟时间少得多）。

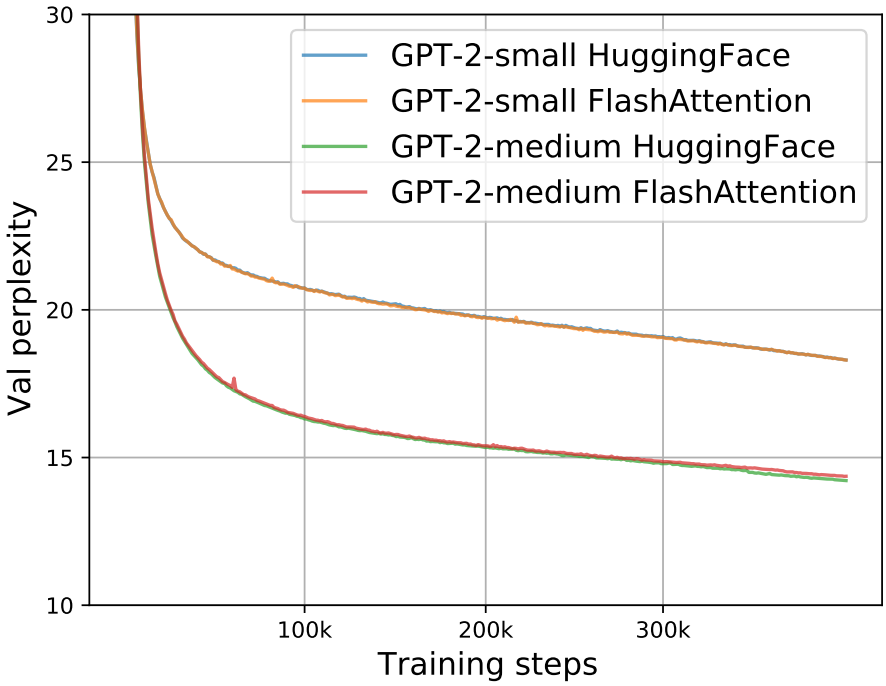

Figure 4: **图 4（论文原图）**。使用两种实现训练 GPT-2 small/medium 的验证困惑度曲线。确认 FlashAttention 产生与 HuggingFace 基线实现相同的验证曲线。

图 5 解读

此图展示了 A100 GPU 上 FlashAttention 在不同配置下的加速效果。图中包含多个子图，分别对应不同的组合（有/无 dropout、有/无 masking）：

- 在所有配置下，加速比通常在 **2–4 倍**之间。
- **使用 dropout 和 masking 时加速更大**——因为 FlashAttention 通过内核融合将这些逐元素操作也一起做了，避免了额外的 HBM 读写。标准实现中，dropout 和 masking 需要分别读写注意力矩阵。
- 随着序列长度增加，加速比通常增大——因为 $N \times N$ 矩阵更大，IO 节省更显著。

图 6 解读

当头维度从 64 增加到 128 时：

- 每个块需要更多内存（$B_c \times d$ 更大），所以必须使用更小的块大小来适配 SRAM。
- 因此**加速比有所下降**，但在使用因果掩码（causal mask）时仍可达到**约 3 倍**加速——因为因果掩码使得约一半的块为零，可以直接跳过。
- 这与 IO 复杂度 $\Theta(N^2d^2/M)$ 的理论一致：d 增大时，IO 复杂度增大，加速比下降。

图 7 解读

RTX 3090 上的加速比（**2.5–4.5 倍**）**略高于 A100**。这是因为：

- RTX 3090 的 HBM 带宽（约 900 GB/s）**低于** A100（约 1.5 TB/s）。

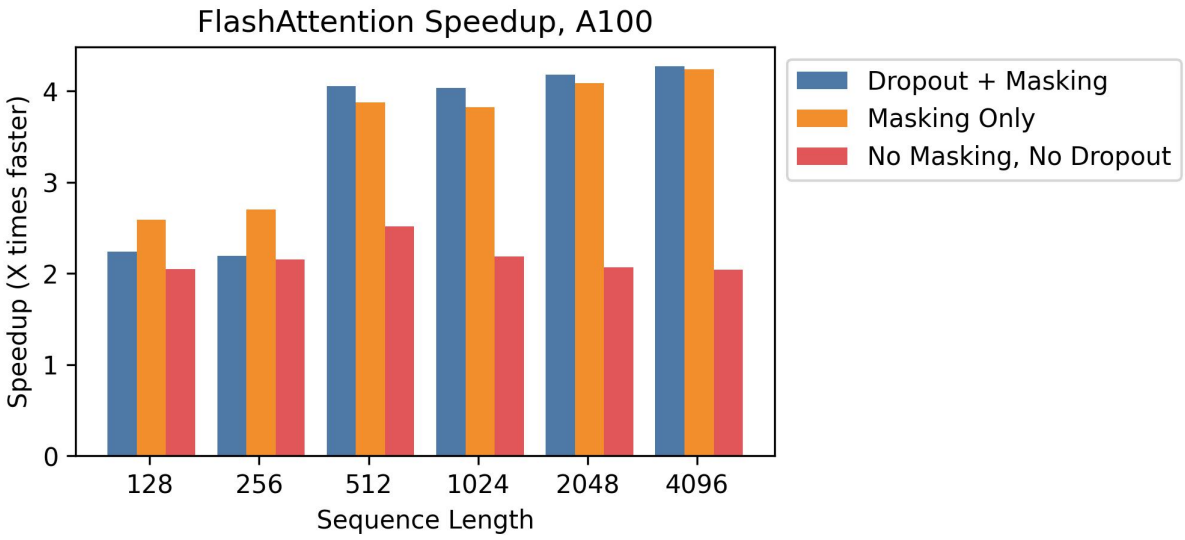

Figure 5: **图 5（论文原图）**。A100 GPU 上不同序列长度的加速比（相对于标准 PyTorch 注意力）。批量大小 8，头维度 64，12 个注意力头。

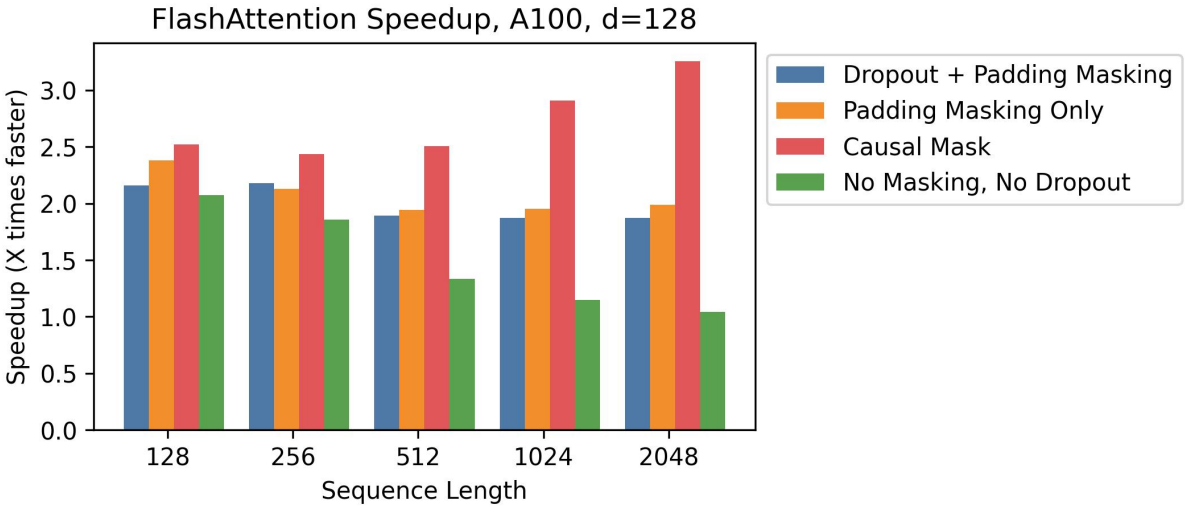

Figure 6: **图 6（论文原图）**。A100 GPU 上头维度 128 时的加速比。批量大小 16，12 个注意力头。

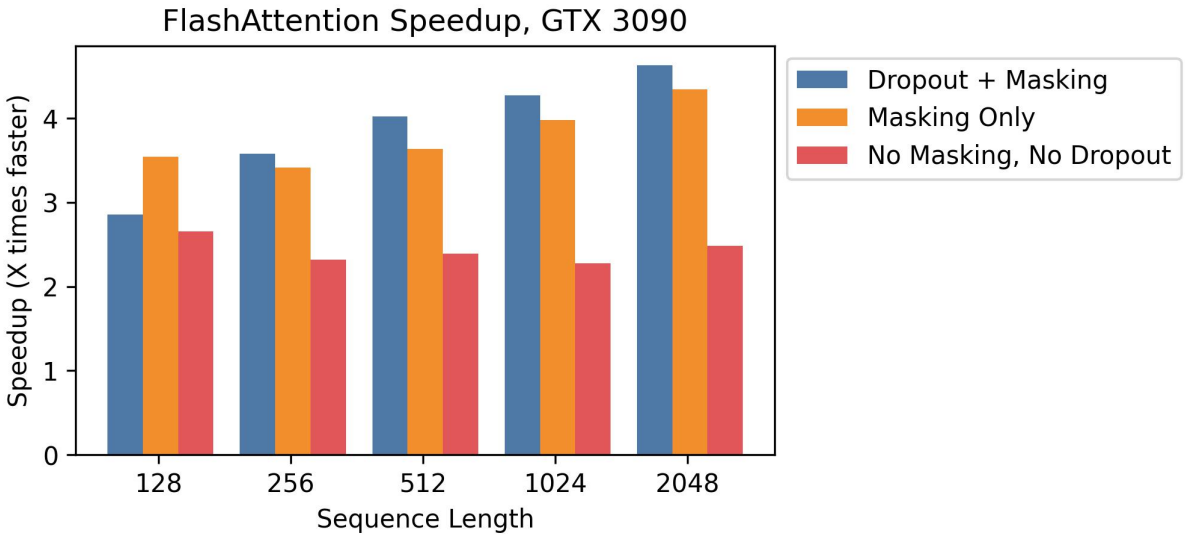

Figure 7: **图 7（论文原图）**。RTX 3090 GPU 上不同序列长度的加速比。批量大小 12，12 个注意力头。

- HBM 带宽越低，内存受限操作的瓶颈越严重，FlashAttention 减少 HBM 访问带来的收益就越大。
- 这验证了理论预测：在内存瓶颈更严重的硬件上，FlashAttention 的加速效果更明显。

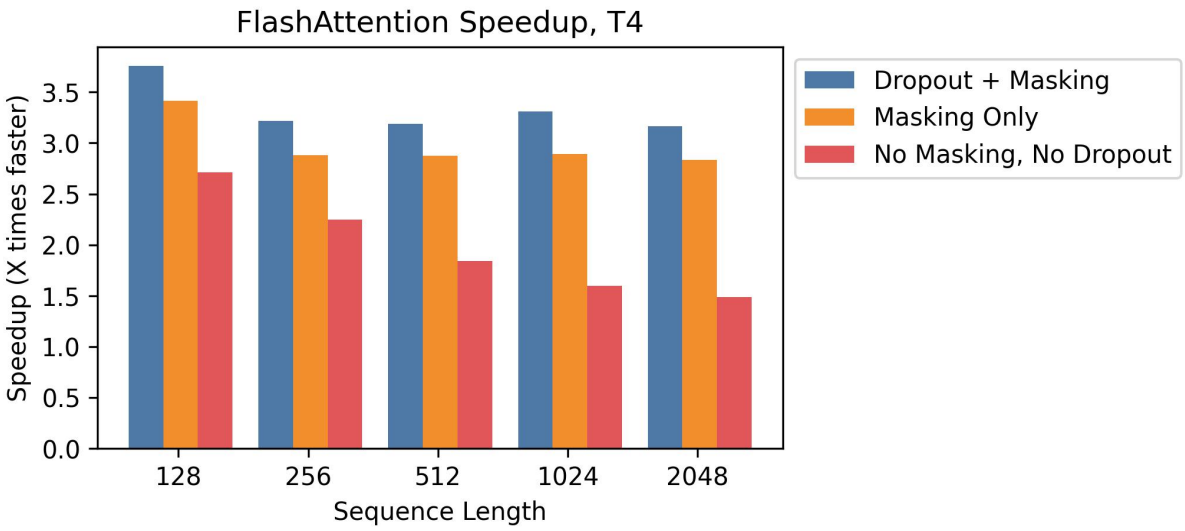

Figure 8: **图 8a（论文原图）**。T4 GPU 上不同序列长度的加速比（前向 + 反向传播）。

图 8a/8b 解读

T4 GPU 是常用的**推理**硬件，其特点是 SRAM 比 A100 更小：

- 由于 SRAM 更小，块大小必须更小 ⇒ 遍历次数更多 ⇒ 加速比低于 A100。

Figure 9: **图 8b（论文原图）**。T4 GPU 上不同序列长度的加速比（仅前向传播，适用于推理场景）。

- 这与 IO 复杂度 $\Theta(N^2d^2/M)$ 一致：M 更小时，IO 复杂度更高，FlashAttention 的优势相对减小。
- 尽管如此，仍然观察到 **1.5–3 倍**的加速。
- 图 8b 单独展示前向传播加速（推理时不需要反向传播），T4 上加速更为明显，因为推理场景不受重计算影响。

12 附录 F：参考文献一览表

以下表格列出论文中引用的主要文献，包括其核心内容和在本文中的引用原因。

文献	主要内容	在本文中的引用原因
Vaswani et al., 2017 (Transformer)	提出了基于自注意力的 Transformer 架构	本文的研究对象，FlashAttention 优化的正是其中的注意力模块
Brown et al., 2020 (GPT-3)	展示了超大规模语言模型的 few-shot 能力	说明 Transformer 规模在不断增长，长序列需求日益迫切
Tay et al., 2020 (LRA)	提出长程竞技场基准，包含五个长序列任务	本文在 LRA 上进行实验，Path-X 是其中最难的任务
Aggarwal & Vitter, 1988	IO 复杂度分析的经典工作	本文 IO 复杂度分析的理论基础
Kitaev et al., 2020 (Reformer)	使用局部敏感哈希的稀疏注意力	近似注意力的代表性基线
Wang et al., 2020 (Linformer)	通过低秩投影近似注意力矩阵	近似注意力的代表性基线
Choromanski et al., 2020 (Performer)	使用随机特征近似 softmax 核	近似注意力的代表性基线
Katharopoulos et al., 2020	将注意力线性化以降低复杂度	线性注意力的代表性基线
Beltagy et al., 2020 (Longformer)	结合局部窗口和全局注意力的长文档 Transformer	组合稀疏 + 局部注意力的代表性方法
Zaheer et al., 2020 (BigBird)	结合随机、局部和全局稀疏注意力	组合稀疏注意力的代表性方法
Chen et al., 2021 (Scatterbrain)	统一稀疏和低秩注意力近似	说明之前的组合方法
Jia et al., 2018/2021	剖析 GPU 内存层次结构和带宽	提供 GPU 硬件参数（HBM 带宽、SRAM 大小等）
Williams et al., 2009 (Roofline)	提出算术强度与 Roofline 分析模型	分析操作是计算受限还是内存受限的框架
Ivanov et al., 2021	分析深度学习操作中的数据移动成本	证明 Transformer 操作多数是内存受限的
Milakov & Gimelshein, 2018	在线/增量 softmax 算法	FlashAttention 分块 softmax 技术的直接来源
Rabe & Staats, 2021	内存高效注意力（不存储 N^2 矩阵）	FlashAttention 与其对比，FlashAttention 进一步优化了 IO 次数
Griewank & Walther, 2008	梯度检查点的经典参考	FlashAttention 的重计算策略是选择性梯度检查点
Chen et al., 2016	亚线性内存成本的深度网络训练	梯度检查点在深度学习中的应用
Radford et al., 2019 (GPT-2)	OpenAI 的 GPT-2 语言模型	本文使用 GPT-2 作为主要的实验基准
Devlin et al., 2019 (BERT)	双向 Transformer 预训练模型	本文在 BERT-large 训练上进行速度基准测试
Mattson et al., 2020 (MLPerf)	机器学习训练性能基准	FlashAttention 打破了 MLPerf 1.1 BERT 训练速度记录

文献	主要内容	在本文中的引用原因
Shoeybi et al., 2019 (Megatron-LM)	Nvidia 的大模型训练框架	高度优化的 GPT-2 训练基线
Wolf et al., 2020 (HuggingFace)	广泛使用的 Transformers Python 库	GPT-2 训练的常用基线实现
Dai et al., 2022	重新审视长文档分类中的 Transformer	本文使用其长文档分类实验设置
Johnson et al., 2016 (MIMIC-III)	ICU 病人出院总结数据集	长文档分类实验的数据集之一
Chalkidis et al., 2019/2021 (ECtHR)	欧洲人权法院案例数据集	长文档分类实验的数据集之一
Gu et al., 2022 (S4)	结构化状态空间模型	处理长序列的替代架构，用于对比
Child et al., 2019 (Sparse Transformer)	使用固定稀疏模式的长序列 Transformer	稀疏注意力基线和稀疏度设置参考
Dao et al., 2019/2020/2022a	蝶形矩阵的学习和表达能力	块稀疏 FlashAttention 使用蝶形稀疏模式
Frankle & Carlin, 2018 (Lottery Ticket)	彩票假说：大网络含高效子网络	块稀疏 FlashAttention 可看作固定彩票
Ragan-Kelley et al., 2013 (Halide)	图像处理的调度-算法分离编译器	期望未来工作能类似地将高级注意力描述编译为 IO 感知实现
Feydy et al., 2020; Charlier et al., 2021 (KeOps)	通过减少内存读写加速核操作	类似的 IO 优化方法在核方法中的应用
Woodruff, 2004	流式算法的最优复杂度	IO 复杂度下界证明中引用的技术
Dosovitskiy et al., 2020 (ViT)	Vision Transformer	说明 Transformer 也广泛用于视觉